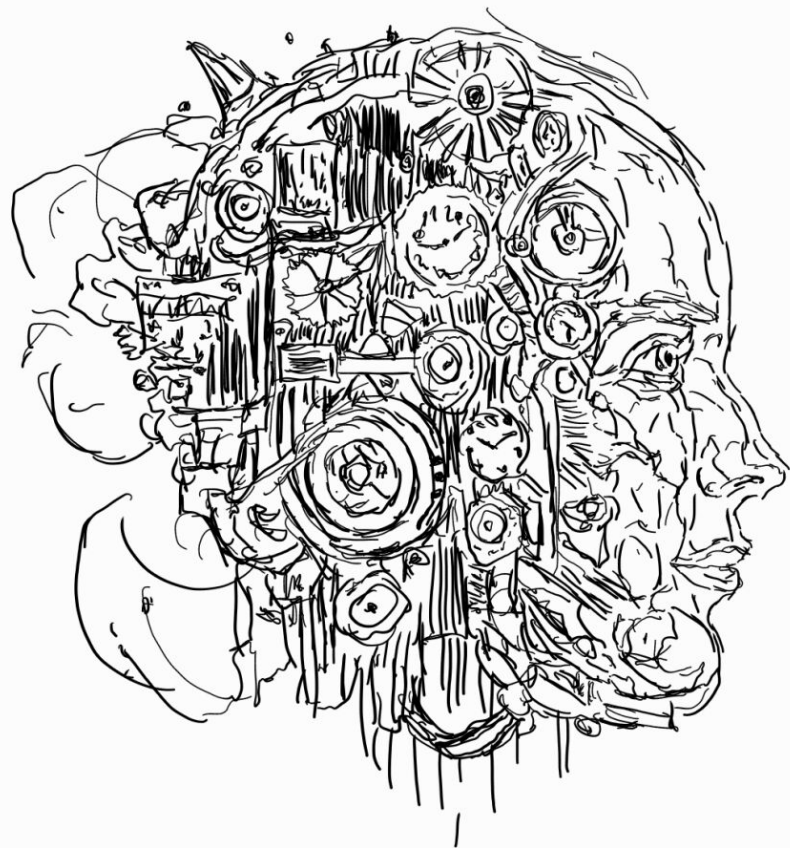


Introduction to Machine Learning

Course summary



Contents

1	Introduction	4
1.1	Idea	4
1.2	Types of Learning	4
1.2.1	Supervised Learning	4
1.2.2	Unsupervised Learning	5
1.2.3	Other types of learning	6
1.3	Machine Learning Pipeline	6
1.3.1	Function Class	7
1.3.2	Training Loss	7
1.3.3	Optimization	8

2	Linear Regression	9
2.1	Simple Linear regression in 1D	9
2.2	Linear regression for general D	9
3	Non-linear Regression	13
4	Optimization	13
4.1	Gradient Descent	13
4.2	Stopping Conditions	15
4.3	Convergence for Linear Regression	15
4.4	Speeding up Gradient Descent	15
4.5	Stochastic Gradient Descent	17
4.6	Model Selection	18
4.7	Test Set	18
4.8	Validation Set	20
4.9	K-fold Cross Validation	20
4.10	Model Complexity	21
5	Bias-variance Tradeoff and Regularization	24
5.1	Bias-variance Tradeoff	24
5.1.1	Bias	24
5.1.2	Variance	26
5.1.3	Conclusion	26
5.2	Regularization	27
5.2.1	Comparing Lasso and Ridge Regression	28
6	Classification	31
6.1	Problem	31
6.2	Binary Classification	31
6.2.1	Exponential and Logistic loss	31
6.2.2	Maximum-Margin and Support Vector Machine	34
6.3	Multiclass classification	36
6.3.1	One-vs-Rest Approach	36
6.3.2	Softmax Function	37
6.3.3	Cross-entropy Loss	38
6.3.4	Asymmetric Losses	39
6.3.5	Receiver Operating Characteristic (ROC)	40
6.4	Trustworthy Generalization	40

7	Kernel Methods	41
7.1	Improving Polynomial Regression	41
7.1.1	Computational Complexity	41
7.1.2	Kernel trick	42
7.2	Other nonlinear methods	44
7.2.1	k -Nearest Neighbor	44
7.2.2	Decision Trees	46
8	Neural Networks	46
8.1	Learning Features	46
9	Clustering	52
9.1	k-means Clustering	53
9.2	k-means++ Clustering	54
10	Unsupervised learning: Dimensionality reduction	54
10.1	Principal Component Analysis	55
11	Probabilistic modeling	59
11.1	Maximum-likelihood estimation (Frequentist)	60
11.2	Maximum-a-posteriori estimation (Bayesian)	61
11.3	Discriminative vs. Generative approach	61
11.4	Gaussian Mixture Models	62

This document is a summary of the Introduction to Machine Learning course that is part of ETH's Computer Science bachelor program. This document was created to summarize and repeat the key learnings of this course. It may contains errors. Franz Schwinn, 06.2024.

1 Introduction

1.1 Idea

Machine learning focuses on the development of algorithms that enable computers to learn and improve from experience without being explicitly programmed. It involves training algorithms on data to recognize patterns, make predictions, or make decisions without explicit instructions.

Applications range from image classification, language processing, recommendation systems, detection of anomalies (e.g. in medical diagnostics), all the way to self driving cars, or drug discovery.



Clarification

Suppose we want to figure out if a patient is at risk of developing skin cancer. As input data we consider time spent outside without proper protection (exposure to UV radiation).

► **Traditional approach:** We let the computer perform tasks based on specific rules that we defined. Example: If UV radiation exposure of the patient is greater than x , then we assume that the patient is at risk. If it is smaller than x , then we assume that the patient is not at risk.

► **Machine Learning approach:** Instead of providing the rules to the computer, we let the computer figure out the rules based on example data that we provide to the computer.

Example: We provide the computer with data (x_i, y_i) where x_i is the UV radiation exposure of patient i and y_i is a boolean value indicating whether patient i developed skin cancer. Instead of specifying a fixed x to decide if the patient is at risk of developing skin cancer (as in the traditional approach), we let the computer efficiently find a $\hat{x}$ that best fits the input data that we provided.

1.2 Types of Learning

1.2.1 Supervised Learning

Definition

Supervised Learning: In supervised learning we provide the computer with a labelled dataset, meaning that each training example is paired with an output label. Using this input data, the computer should learn rules which can be applied to new data. The above mentioned "UV radiation" example is a case of supervised learning (y_i in the input data represents the label).

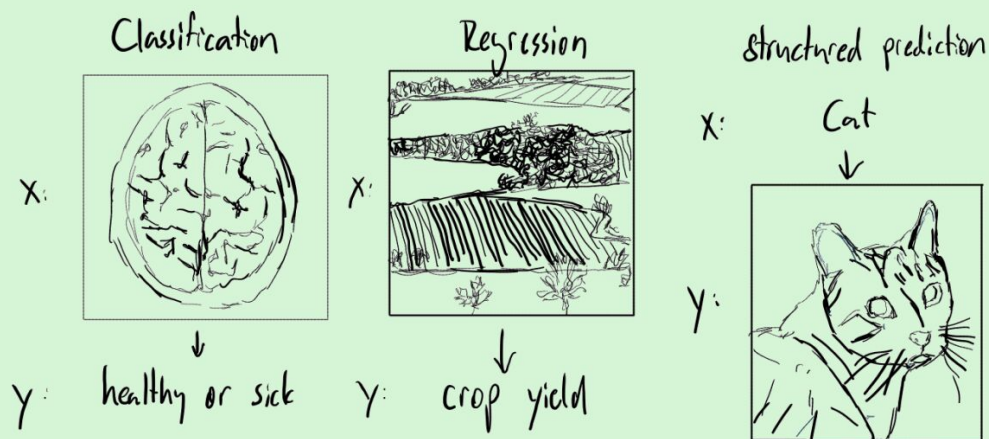
Below are some supervised learning tasks:

► **Classification:** Predict the category to which the given input data belongs to (e.g. "spam" or

"no spam" for emails, or "0", "1", "2", "3", "4" representing cancer stages in images). So we predict a discrete scalar.

► **Regression:** Predict a continuous scalar for a given input (e.g. house prices based on features like location, size, and number of rooms, or estimating the age of a person's photo). So we predict a continuous scalar.

► **Structured Prediction:** Predict complex outputs such as sequences, graphs or images (e.g. language translation, speech recognition, or image segmentation).



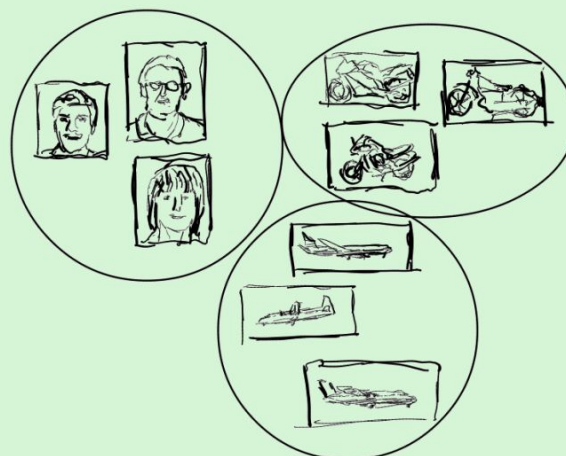
1.2.2 Unsupervised Learning

Definition

Unsupervised Learning: In unsupervised learning the computer learns patterns from unlabelled data. This means that the training data does not contain output data, therefore there is no supervision. It can be used to detect patterns or relationships within the data. Examples are clustering or anomaly detection.

Below are some unsupervised learning tasks:

► **Clustering:** Given a set of images, identify which images belong together.



► **Dimensionality reduction:** The idea is to reduce the number of input variables (dimensions) while keeping as much information as possible. By reducing the dimensions we increase the performance because we process less variables. Typically we are interested in removing the noise or redundancy. So we transform the original high-dimensional data into a lower-dimensional representation while retaining as much relevant information as possible.

► **Generative modeling:** The idea is to learn the underlying probability distribution of the training data in order to generate new data samples similar to the ones in the training set. By this the machine can create new instances of data that is similar to the originally submitted training data.

1.2.3 Other types of learning

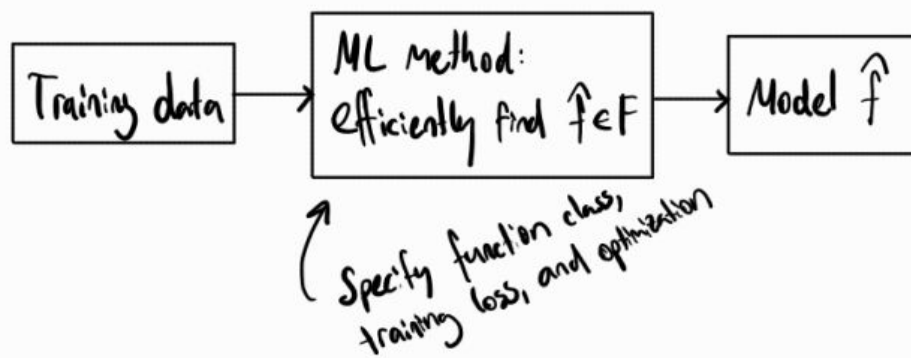
- **Semi-supervised learning:** Learning from labelled and unlabelled data
- **Active learning:** Acquire most informative labels for learning
- **Online / lifelong / continual learning:** Learning from data as it arrives over time
- **Reinforcement learning:** Learning by interacting with an unknown environment

1.3 Machine Learning Pipeline

Suppose we want to use machine learning to estimate the price of a house. If we can find the prices of other houses we can use supervised learning and shape a regression task. We can proceed as shown below:

1. **Find numerical representation for house:** Since we cannot parse a house directly to a computer, we need to describe all properties that we are interested in by numerical values (e.g. location, size, number of bedrooms, age). For this simple example, let's only consider the area of the house.
2. **Collect training data:** We want to collect labeled data (i.e. sizes and prices of other houses).
3. **Learn:** Using the collected training data, we want to efficiently find a function $\hat{f} \in F$ that fits our training data. F is a function class (e.g. linear functions, 2nd degree polynomials, etc.), and $\hat{f}$ is one specific function from F .
4. **Predict:** Finally, we can plug in the numerical representation of our own house into $\hat{f}$ to get an estimation of the price of our house ($\hat{f}(\text{area of our house}) = \text{price estimation of our house}$).

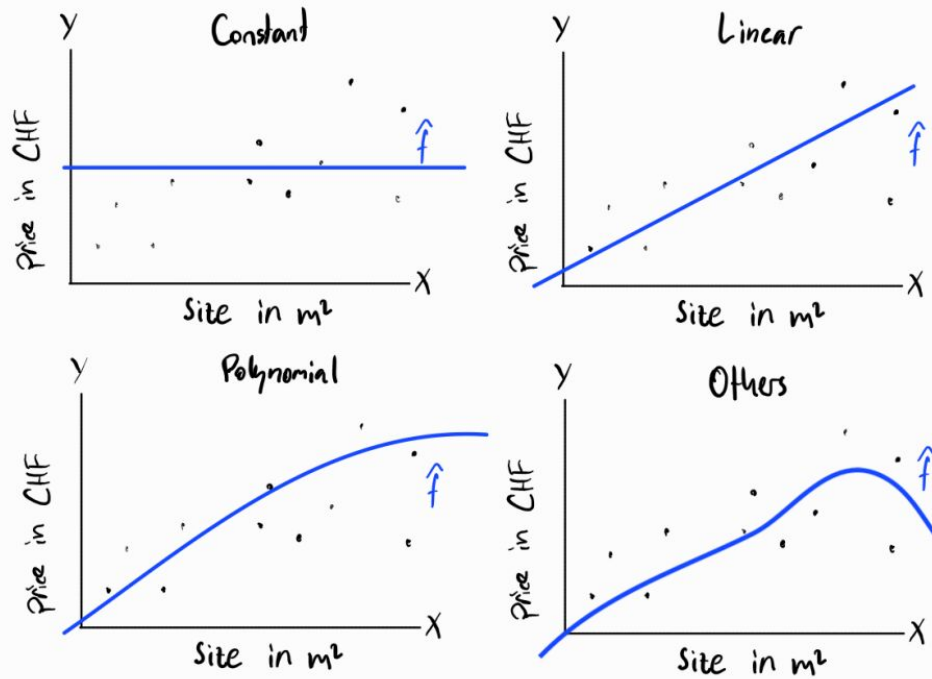
Returning to the previous example of estimating the price of a house, we now only consider the area (x), and the house price (y). Therefore, our training data consists of (x_i, y_i) for the i th house.



But how do we find $\hat{f}$? For this we need to specify the function class F , the training loss, and the optimization.

1.3.1 Function Class

As mentioned before, a function class is a collection of functions that have certain properties.



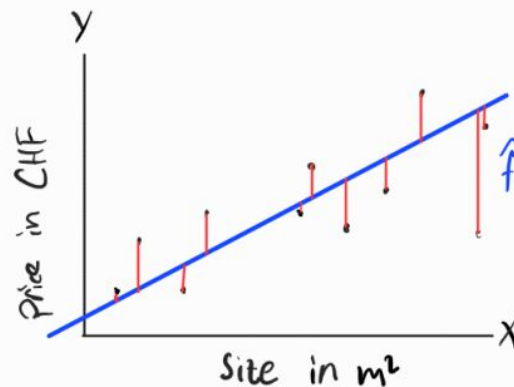
Therefore we first need to think about the shape or form of the function $\hat{f}$ that we are looking for.

1.3.2 Training Loss

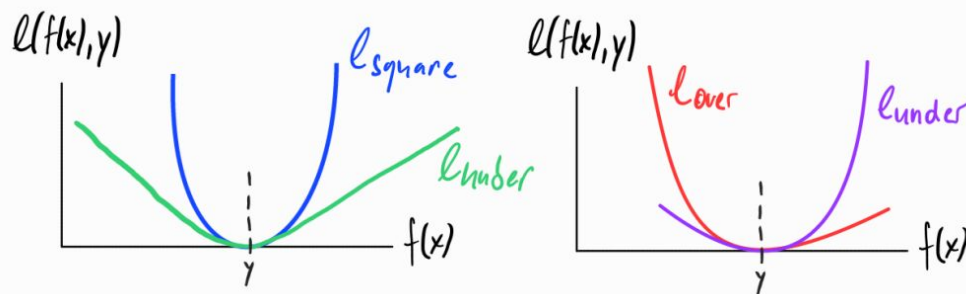
As we want to find a function $\hat{f}$ that best fits our training data, we first have to clarify what "best fit" means. We want to have low training loss, which means that $\hat{f}(x_i)$ should be close to y_i for most points. We define the training loss as

$$L(f) = \frac{1}{n} \sum_{i=1}^n l(f(x_i), y_i)$$

where l is the **pointwise loss function**. It takes in a function $f \in F$ and represents the "closeness" between $f(x_i)$ and y_i for a point (x_i, y_i) . Intuitive loss functions are $|\hat{f}(x_i) - y_i|$ (the absolute distance between the label and the estimated result) or $(\hat{f}(x_i) - y_i)^2$ (squared loss - outliers are more strongly weighted).



Other examples are the Huber loss (weight outliers less than in the square loss method), or the asymmetric loss method (weight over- and underestimation differently).



1.3.3 Optimization

Optimization describes the way we find the minimum of the training loss function (and therefore find the best possible weights). There are several approaches:

Closed Form: This is the most straight forward approach: we directly compute the minimum of the training loss function (e.g. by setting the gradient to zero). But this approach cannot be used if we have training loss functions with many local minima. Finding the global minima by checking all local minima may be inefficient.

Gradient Descent: In this approach we try to find the minimum in an iterative manner. This means that we start at a given point, and iteratively go in the direction of negative gradient (the gradient points in the direction of steepest ascent at a given point - therefore we go in the negative direction of the gradient). This allows us to find a local minimum.

Stochastic Gradient Descent: Instead of computing the exact gradient of the entire dataset at each step, we randomly select a subset of the data and compute the gradient for that subset. This makes the algorithm more performant and helps to avoid local minima.

Momentum: We add momentum to gradient descent. This means that we move in the direction of the negative gradient and the accumulated past gradients (momentum). By considering the momentum, we smooth out the path and accelerate the convergence. Momentum helps to get rid of oscillations and can lead to a faster and more stable descent.

2 Linear Regression

2.1 Simple Linear regression in 1D

For simple linear regression in 1D we choose the function class of all linear functions ($F = \{w_1x + w_2 | w_1, w_2 \in \mathbb{R}\}$, w_1 and w_2 are called weights) and the squared loss approach. Our job is to find the linear function $\hat{f} \in F$ that minimizes the training loss (average squared loss):

$$\begin{aligned}\hat{f} &= \operatorname{argmin}_{f \in F} (L(f)) && / \text{ find function that minimizes loss} \\ &= \operatorname{argmin}_{f \in F} \left(\frac{1}{n} \sum_{i=1}^n (f(x_i) - y_i)^2 \right) && / \text{ definition of loss function}\end{aligned}$$

Since $f = w_1x + w_2$ we can find the correct parameters by

$$\hat{w} := (\hat{w}_1, \hat{w}_2) = \operatorname{argmin}_{w_1, w_2 \in \mathbb{R}} \left(\frac{1}{n} \sum_{i=1}^n (w_1x_i + w_2 - y_i)^2 \right)$$

Theorem

Necessary condition for local optimality: Let $\Omega \subset \mathbb{R}^n$ be open and $f : \Omega \rightarrow \mathbb{R}$ a continuously differentiable function. Then we have:

$$x_0 \in \Omega \text{ is a local minimizer} \implies \nabla f(x_0) = 0$$

Therefore, when searching for a local minimizer, we only need to consider the set of critical points defined as $\{x \in \Omega | \nabla f(x) = 0\}$.

Therefore it follows that the global minimum $\hat{w}$ must satisfy $\nabla_w L(\hat{w}) = 0$. The training loss is $L(w_0, w_1) = \frac{1}{n} \sum_{i=1}^n ((w_1x_i + w_2 - y_i)^2)$. The gradient must be 0.

$$\nabla_w L(\hat{w}_1, \hat{w}_2) = \begin{pmatrix} \nabla_{\hat{w}_1} L(\hat{w}_1, \hat{w}_2) \\ \nabla_{\hat{w}_2} L(\hat{w}_1, \hat{w}_2) \end{pmatrix} = \begin{pmatrix} \frac{2}{n} \sum_{i=1}^n ((w_1x_i + w_2 - y_i)x_i) \\ \frac{2}{n} \sum_{i=1}^n (w_1x_i + w_2 - y_i) \end{pmatrix} \stackrel{!}{=} 0$$

The number of solutions depends on our set of points.

2.2 Linear regression for general D

Instead of just considering one attribute (the size of the house), we want to include other attributes such as distance to city center, age of the property, or number of rooms.

Clarification

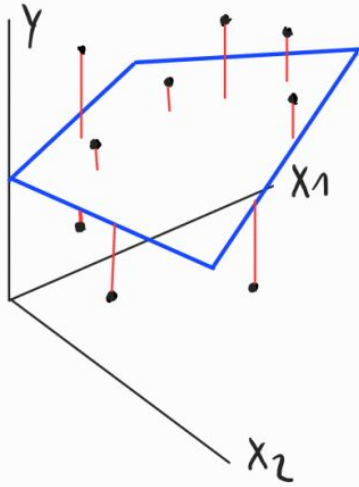
We denote $x_{[i]}$ to describe the i th attribute of an input vector, and x_i to describe the i th input vector. Example attributes for dimensionality $d = 3$:

- $x_{[1]}$ = size in square meters
- $x_{[2]}$ = distance to city center in kilometers
- $x_{[3]}$ = age of the property in years

And input data:

- $x_1 = (100, 3, 40)$
- $x_2 = (200, 6, 70)$
- $x_3 = (50, 1, 3)$

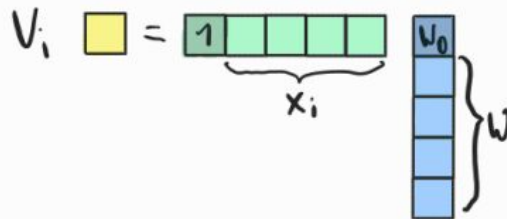
Function class: To model linear regression for general d we pick the function class of all linear functions with d features $F = \{w_0 + \sum_{i=1}^d w_i x_{[i]} | w_0, \dots, w_d \in \mathbb{R}\}$. In vector-notation, this is equal to $\{w_0 + w^T x | w_0 \in \mathbb{R}, (w_1, \dots, w_d) \in \mathbb{R}^d\}$. This can be visualized as a plane.



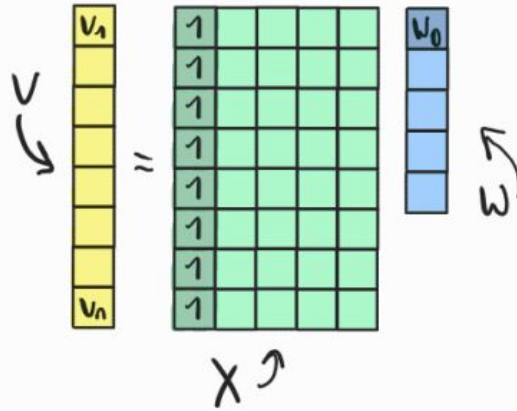
Training loss: We again consider the squared loss method. Therefore, the pointwise loss function is $(f(x_i) - y_i)^2$ (for given point i) and the average loss function over all points is $L(f) = \frac{1}{n} \sum_{i=1}^n (f(x_i) - y_i)^2$. We look for the optimal function $\hat{f} \in F$ such that $\hat{f} = \operatorname{argmin}_{f \in F} L(f) = \operatorname{argmin}_{f \in F} (\frac{1}{n} \sum_{i=1}^n (f(x_i) - y_i)^2)$. We now want to represent this equation using vector-notation. Since f is of the form $f(x) = w_0 + w^T x$ we can also minimize over the weights (so the scalar w_0 and the vector w):

$$(\hat{w}_0, \hat{w}) = \operatorname{argmin}_{w_0 \in \mathbb{R}, w \in \mathbb{R}^d} \frac{1}{n} \sum_{i=1}^n (w_0 + w^T x_i - y_i)^2$$

Let's define v_i as the blue expression, so $v_i := w_0 + w^T x_i$. This can be visualized as follows:



The entire vector v is therefore:



Going back to our training loss function:

$$\begin{aligned}
 L(w_0, w) &= \frac{1}{n} \sum_{i=1}^n (w_0 + w^T x_i - y_i)^2 && / \text{ loss function} \\
 &= \frac{1}{n} \sum_{i=1}^n (v_i - y_i)^2 && / \text{ definition of } v_i \\
 &= \frac{1}{n} \|v - y\|_2^2 && / \text{ by def of 2-norm (squared to get rid of root)} \\
 &= \frac{1}{n} \|\mathbf{1}w_0 + Xw - y\|_2^2 && / v = \mathbf{1}w_0 + Xw \text{ (}\mathbf{1} \text{ is a vector with only 1s)}
 \end{aligned}$$

For simplicity reasons, let's set $w_0 = 0$.

$$\begin{aligned}
 L(0, w) &= \frac{1}{n} \|Xw - y\|_2^2 && / w_0 = 0 \\
 &= \frac{1}{n} (\|Xw\|_2^2 - 2(Xw)^T y + \|y\|_2^2) && / \|a + b\|_2^2 = \|a\|_2^2 + 2a^T b + \|b\|_2^2 \text{ (see clarification)} \\
 &= \frac{1}{n} ((Xw)^T Xw - 2(Xw)^T y + y^T y) && / \|a\|_2^2 = a^T a \\
 &= \frac{1}{n} (w^T X^T Xw - 2(w^T X^T)y + y^T y) && / (ab)^T = b^T a^T
 \end{aligned}$$

Clarification

Why is $\|a + b\|_2^2 = \|a\|_2^2 + 2a^T b + \|b\|_2^2$?

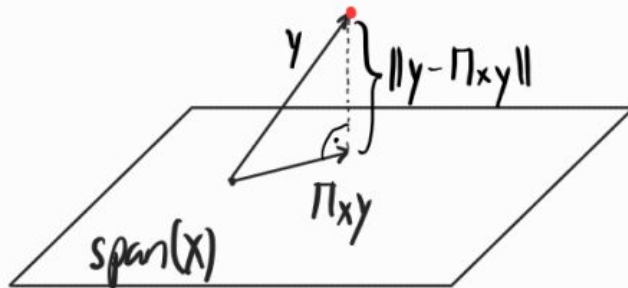
$$\begin{aligned}
 \|a + b\|_2^2 &= \sqrt{\sum_{i=1}^n (a_i + b_i)^2}^2 && / \text{ definition of 2-norm} \\
 &= \sqrt{(a + b) \cdot (a + b)}^2 && / \text{ formulate to multiplication} \\
 &= (a + b) \cdot (a + b) && / \text{ root cancelled out} \\
 &= (a + b)^T (a + b) && / \text{ definition of dot product} \\
 &= (a^T + b^T)(a + b) && / (a + b)^T = (a^T + b^T) \\
 &= a^T a + a^T b + b^T a + b^T b && / \text{ expand} \\
 &= a \cdot a + a \cdot b + b \cdot a + b \cdot b && / \text{ definition of dot product} \\
 &= a \cdot a + 2(a \cdot b) + b \cdot b && / \text{ commutativity of dot product} \\
 &= \|a\|_2^2 + 2(a^T b) + \|b\|_2^2 && / \text{ definition of 2-norm}
 \end{aligned}$$

Now we take the gradient with respect to w :

$$\begin{aligned}
 \nabla_w L(0, w) &= \frac{1}{n} (2X^T X w - 2X^T y) && / \text{ take gradient} \\
 &= \frac{2}{n} X^T X w - \frac{2}{n} X^T y && / \text{ expand} \\
 &\stackrel{!}{=} 0 && / \text{ gradient must be 0}
 \end{aligned}$$

$$\begin{aligned}
 \frac{2}{n} X^T X w - \frac{2}{n} X^T y = 0 &\implies \frac{2}{n} X^T X w = \frac{2}{n} X^T y && / + \frac{2}{n} X^T y \\
 &\implies X^T X w = X^T y && / \cdot \frac{n}{2} \\
 &\implies w = (X^T X)^{-1} X^T y && / \text{ if } X^T X \text{ is invertible}
 \end{aligned}$$

Geometric interpretation: We can also find a geometric interpretation which may be more intuitive. We want to find $\hat{w}$:



The set of all possible Xw is equal to the image of X (so $\text{span}(X) = \{Xw | w \in \mathbb{R}^d\}$). The closest point to y on $\text{span}(X)$ is the orthogonal projection of y onto $\text{span}(X)$. We can denote it as $\Pi_X y$.

We now want to find the $\hat{w}$ that satisfies $X\hat{w} = \Pi_X y$. We know that whatever $w \in \mathbb{R}^d$ we pick, it will always

be orthogonal to $y - X\hat{w}$.

$$\begin{aligned}
 y - X\hat{w} \perp Xw \text{ for all } w &\implies (y - X\hat{w}) \cdot Xw = 0 \text{ for all } w && / \text{ dot product of orthogonal vectors is 0} \\
 &\implies (y - X\hat{w})^T Xw = 0 \text{ for all } w && / \text{ definition of dot product} \\
 &\implies uw = 0 \text{ for all } w \text{ and } u = (y - X\hat{w})^T X && / \text{ define } u = (y - X\hat{w})^T X \\
 &\implies u = 0 \\
 &\implies (y - X\hat{w})^T X = 0 && / \text{ by definition of } u \\
 &\implies y^T X - (X\hat{w})^T X = 0 && / \text{ linear algebra rules} \\
 &\implies y^T X = (X\hat{w})^T X && / + (X\hat{w})^T X \\
 &\implies X^T y = X^T X \hat{w} && / a^T b = b^T a \\
 &\implies (X^T X)^{-1} X^T y = \hat{w} && / \text{ if } X^T X \text{ is invertible}
 \end{aligned}$$

This is the same result as calculated using the gradient!

3 Non-linear Regression

We can also choose a function class of non-linear functions to fit our data points (e.g. set of polynomials of degree $p-1$: $F = \{w_0 + w_1x + \dots + w_{p-1}x^{p-1}\}$).

These non-linear functions can be represented as $f(x) = \sum_{i=1}^p w_i \phi_i(x)$ where $\phi(x) = (\phi_1(x), \dots, \phi_p(x))$ is defined as the **feature vector**. So $f(x)$ is a linear combination of the feature vectors. In the example of polynomials, $\phi(x) = (1, x, \dots, x^{p-1})$. But we could also come up with a trigonometric basis $\phi(x) = (1, \sin(2\pi x), \dots, \sin(2\pi(p-1)x))$ or anything else.

We are interested in finding the best weights (consider squared loss again):

$$\begin{aligned}
 \hat{w} &= \operatorname{argmin}_{w \in \mathbb{R}^p} L(w) \\
 &= \operatorname{argmin}_{w \in \mathbb{R}^p} \left(\frac{1}{n} \sum_{i=1}^n (y_i - w^T \phi(x_i))^2 \right) \\
 &= \|y - \Phi w\|^2
 \end{aligned}$$

4 Optimization

How do we find the minimum of a training loss function in the most efficient way?

4.1 Gradient Descent

Most of the time we cannot simply compute the minimum of the training loss function (so we cannot follow the closed-form approach). The reason is that many machine learning models have a very high number of parameters (often in the millions or even billions). The loss function's landscape in such a high-dimensional space is incredibly complex. Analytically solving for the minimum of such functions is too inefficient.

Therefore we have the below iterative algorithm to find a local minimum of a differentiable training loss function.

Definition

General iterative algorithm to minimize $L(w)$:

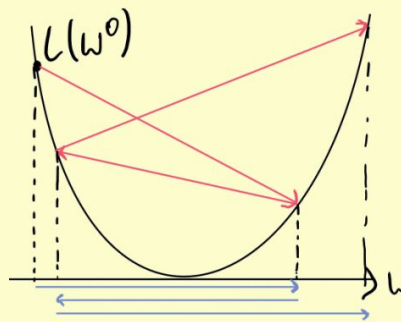
1. Start at an initial w^0
2. At each step t , calculate $w^{t+1} = w^t - \eta \nabla_w L(w^t)$
3. Stop at certain condition (e.g. $|L(w^{t+1}) - L(w^t)| \leq \epsilon$)

4. Output $\hat{w} = w^t$ **Clarification**

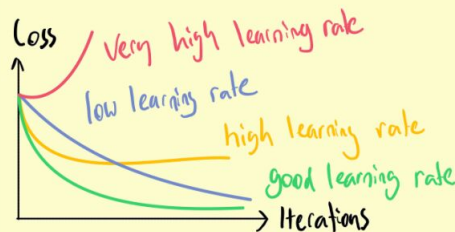
1. Start at an initial w^0 : To begin the iterative approach, we first need to choose a starting point w^0 . This point can be chosen at random or in a certain range (in case we have some vague idea where the minimum might be).

2. At each step t , calculate $w^{t+1} = w^t - \eta \nabla_w L(w^t)$: How did we come up with this formula? Let's say that in every iteration we want to move $\tilde{\eta}$ units (called step size) in a certain direction v (v is a unit vector, so $\|v\| = 1$). This gives us $w^{t+1} = w^t + \tilde{\eta}v$.

- **Step size:** It would be great if the step size $\tilde{\eta}$ is not constant. If it was constant, then we might overshoot the minimum of the function and fail to converge.



Having a constant step size may also result in slow convergence. Therefore we set $\tilde{\eta} = \eta \|\nabla_w L(w^t)\|$ where η is a constant. If we are at some steep point $L(w^t)$, the 2-norm of the gradient is large ($\|\nabla_w L(w^t)\|$ is large) resulting in a large step size. But if we are at some flat point $L(w^t)$, the 2-norm of the gradient is small, resulting in a small step size.



- **Direction:** We know that the gradient points to the direction of steepest ascent. As we want to find a minimum of the loss function, we need to navigate in negative direction of the gradient. Therefore we have $v = -\frac{\nabla_w L(w^t)}{\|\nabla_w L(w^t)\|}$ (we divide by the length of the gradient to normalize it - v should be a unit vector).

This gives us:

$$\begin{aligned}
 w^{t+1} &= w^t + \tilde{\eta}v && / \text{ add step size times some unit vector} \\
 &= w^t + (\eta \|\nabla_w L(w^t)\|) \left(-\frac{\nabla_w L(w^t)}{\|\nabla_w L(w^t)\|} \right) && / \text{ step size and direction as shown above} \\
 &= w^t + (\eta) (-\nabla_w L(w^t)) && / \text{ the norm cancels out} \\
 &= w^t - \eta \nabla_w L(w^t)
 \end{aligned}$$

4.2 Stopping Conditions

When do we stop the iterations? Below are three approaches:

► **Convergence criterion:** Stop when the difference between the last two iterations falls below a certain constant ϵ (so $|L(w^{t+1}) - L(w^t)| \leq \epsilon$). This indicates that next iterations are unlikely to further improve the solution.

► **Gradient norm:** Stop when the norm of the gradient falls below a certain constant ϵ (so $|\nabla L(w^t)| \leq \epsilon$).

► **A priori termination:** Abort the iterations after n steps, no matter how close we are to the solution. There are more stopping conditions - each with its own up and downsides.

4.3 Convergence for Linear Regression

How can we prove that our iteration converges to $\hat{w} = \operatorname{argmin}_{w \in \mathbb{R}^d} L(w)$?

Assume we begin our iteration at a certain point w^t . Now we need to show that the current distance to $\hat{w}$ is greater than the distance of the next iteration w^{t+1} to $\hat{w}$ (formally: $\|w^{t+1} - \hat{w}\|_2 \leq \|w^t - \hat{w}\|_2$).

$$\begin{aligned} \|w^{t+1} - \hat{w}\|_2 &= \|w^t - \eta \nabla L(w^t) - \hat{w}\|_2 && / w^{t+1} = w^t - \eta \nabla L(w^t) \text{ as shown previously} \\ &= \|w^t - \eta(X^T X w^t - X^T y) - \hat{w}\|_2 && / \nabla L(w^t) = X^T X w^t - X^T y \text{ for } n = 2 \text{ as shown above} \\ &= \|w^t - \eta(X^T X w^t - X^T X \hat{w}) - \hat{w}\|_2 && / X^T X \hat{w} = X^T y \text{ as shown previously} \\ &= \|(I - \eta X^T X)(w^t - \hat{w})\|_2 \end{aligned}$$

Definition

Operator norm: For any matrix A :

$$\|A\|_{op} = \sup_z \frac{\|Az\|_2}{\|z\|_2}$$

So $\|A\|_{op}$ is the largest factor by which A can stretch a vector z . Or alternatively, $\|A\|_{op}$ is the length of the largest vector we get by plugging in all possible unit vectors z .

Note that for all vectors z' : $\frac{\|Az'\|_2}{\|z'\|_2} \leq \|A\|_{op}$. By multiplying both sides by $\|z'\|_2$ we get $\|Az'\|_2 \leq \|A\|_{op} \|z'\|_2$.

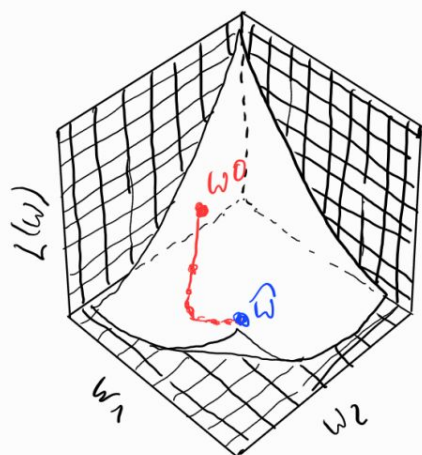
Above we derived:

$$\begin{aligned} \|w^{t+1} - \hat{w}\|_2 &= \|(I - \eta X^T X)(w^t - \hat{w})\|_2 && / \text{from above} \\ &= \|(I - \eta X^T X)(w^t - \hat{w})\|_2 && / \text{let } A = (I - \eta X^T X) \text{ and } z' = (w^t - \hat{w}) \\ &\leq \|I - \eta X^T X\|_{op} \|w^t - \hat{w}\|_2 && / \|Az'\|_2 \leq \|A\|_{op} \|z'\|_2 \\ &= \rho \|w^t - \hat{w}\|_2 && / \text{let } \rho = \|I - \eta X^T X\|_{op} \end{aligned}$$

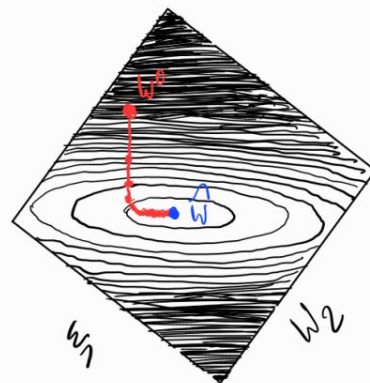
This means we have $\|w^{t+1} - \hat{w}\|_2 \leq \rho \|w^t - \hat{w}\|_2$. When $\rho < 1$ then we know that $\|w^t - \hat{w}\|_2 > \|w^{t+1} - \hat{w}\|_2$. This can guarantee this by choosing the correct step size η .

4.4 Speeding up Gradient Descent

If the contour lines around the optimum are circles, gradient descent converges faster than if the contour lines around the optimum are stretched ellipses.

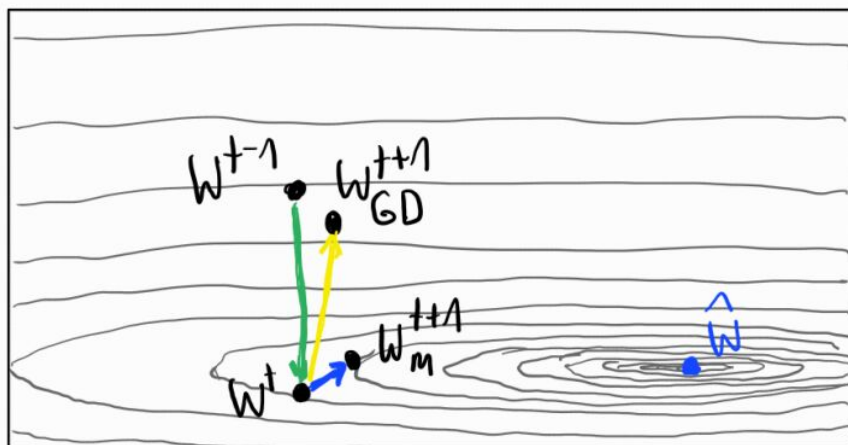


3d loss function plot



2d contour plot

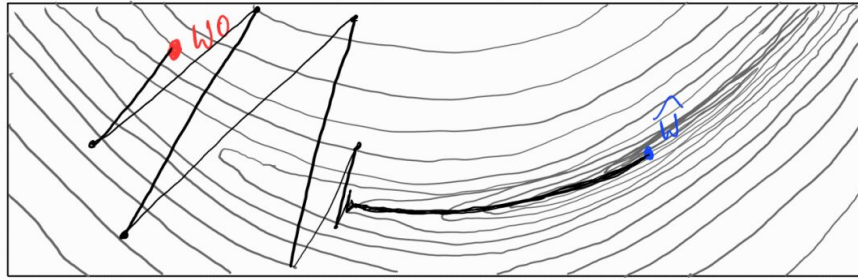
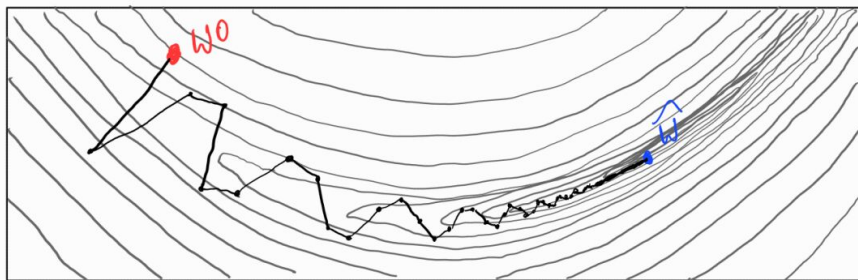
The idea is to introduce momentum to GD. Therefore we modify GD as follows: $w^{t+1} = w^t + \beta(w^t - w^{t-1}) - \eta \nabla L(w^t)$ (with β we can control the momentum).



$$\beta \approx 0,8$$

This speeds up GD in flat areas and additionally dampens oscillations.

GD:

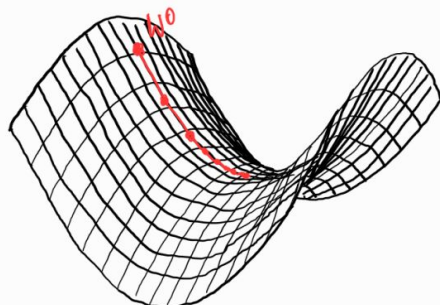
GD with momentum $\beta=0,8$:

4.5 Stochastic Gradient Descent

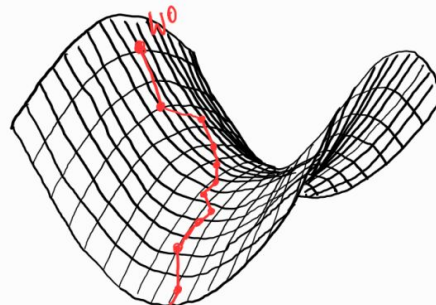
Using GD can be computationally expensive, especially for large datasets (large number of training points n). The memory required is $O(nd)$ and the computation time for one iteration $O(n \cdot \text{time to compute } \nabla L(w))$. In SGD, we partition all data points into i small mini-batches $S_1, \dots, S_i$. For each iteration we consider the points in the mini-batch, instead of all points (instead of $\nabla L(w)$ we use $\nabla L_S(w) = \frac{1}{|S|} \sum_{i \in S} \nabla l(y_i, f(x_i))$). This means that our loss function is not the same for every iteration, but it is just an approximation of the training loss function. This results in inaccuracies, but because SGD is so much faster we can perform more iterations.

As mentioned, SGD works great for large datasets. Additionally, the fact that only the points in the mini-batch are considered results in some variation. This can help to escape local minima and saddle points more easily.

GD:



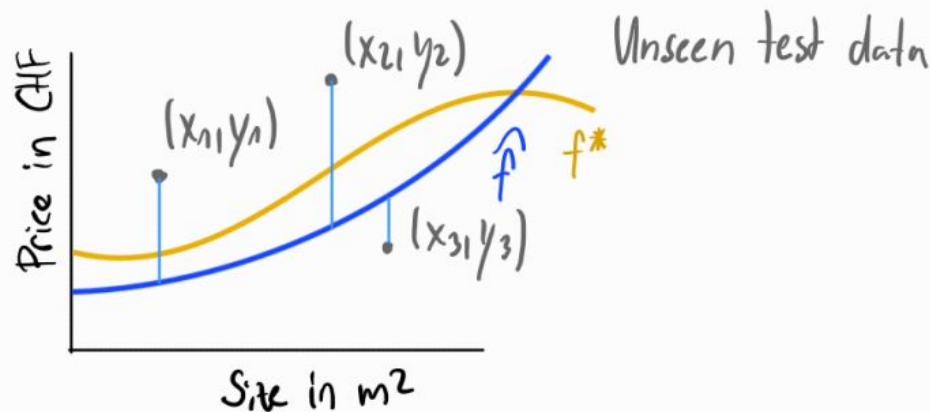
SGD:



4.6 Model Selection

The machine learning engineer must find the best possible model (function class, training loss method and optimization method) such that it accurately captures the underlying patterns in the data and generalizes well to unseen data. Therefore we need to find a way to measure how good our estimation $\hat{f}$ is. We want to get a sense for how well $\hat{f}$ does in predicting unseen test samples.

We assume that there is a ground truth function f^* that we would be able to compute if we had all possible data points available, and without any measure errors. A good model therefore should have a low estimation error $l(f^*(x), \hat{f}(x)) = (f^*(x) - \hat{f}(x))^2$. Sadly we cannot compute the estimation error because we don't know how the ground truth function f^* looks like. Therefore, a good model $\hat{f}$ should have a low prediction error on unseen test input (a subset of the data that we collected).



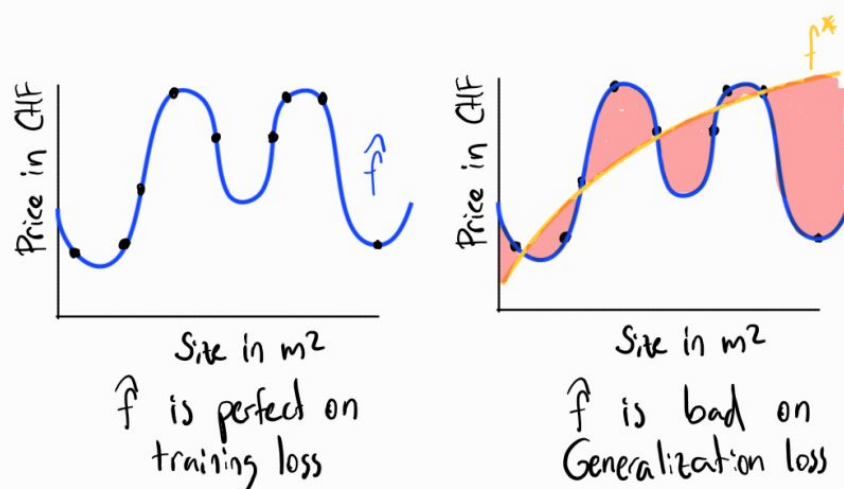
Definition

Training error/loss: Training error refers to the loss that $\hat{f}$ incurs on the data it was trained on. The training error gives an indication of how well the model fits the training data. A low training error suggests that the model has learned the patterns in the training data well.

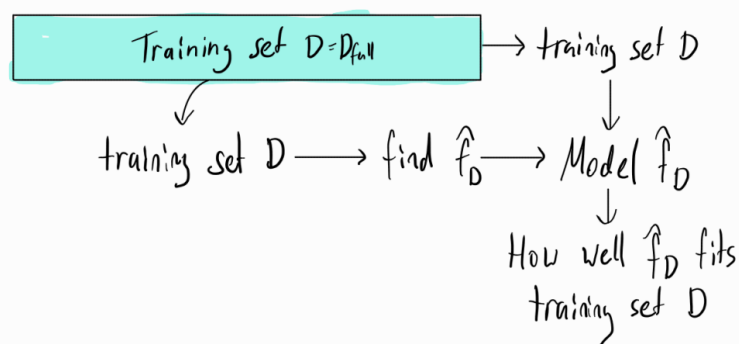
Generalization error/loss: Generalization error (or test error), is the loss on new, unseen data. It is calculated by evaluating the model's performance on a separate dataset called the test dataset, which was not used during training. The generalization error measures how well the model can generalize its predictions to new, unseen examples. It provides an estimate of how well the model will perform in real-world applications.

4.7 Test Set

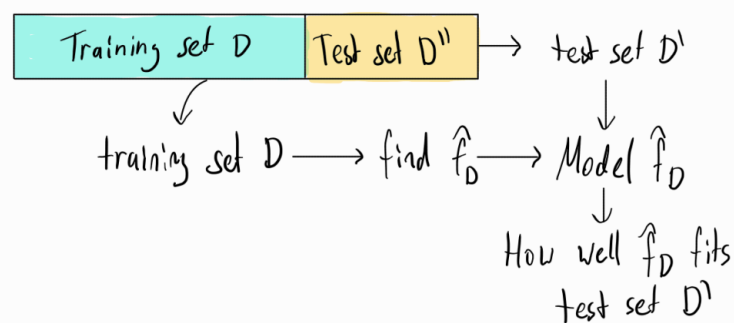
Usually the training loss is too optimistic because it optimizes for the training data which does not necessarily reflect how well the model generalizes to unseen data. Below is a visualization of overfitting. Intuitively this means that the model learned to memorize the training data instead of generalizing it for unseen data.



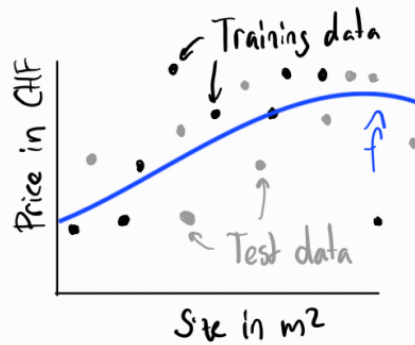
So far we used all data points for training. This can lead to overfitting and additionally we are not able to measure how well the model performs on unseen data.



The main idea is to hold out part of the available data (we call this held-out set "test data") and use the rest of the data points for training.



We train on all data that is not in the test set and use the test data only for approximating the generalization error.



When training the model on the training set D , we get the estimation $\hat{f}$. The generalization error can now be calculated similarly as the training loss, but only over the data in the test set D'' : $\frac{1}{|D''|} \sum_{(x,y) \in D''} l(\hat{f}(x), y)$. Usually, the more data we have the larger the test data set becomes (in relative terms).

Note: In some cases it makes sense to split the data randomly (e.g. with housing prices example), whereas in other cases data must be split in chunks (e.g. time related data like temperature measurements).

4.8 Validation Set

Additionally to the test data, we can hold out another subset of points in the training data to use it as a pre-validation for a model. Intuitively, it's a practice exam for the model before the final test against the test data. This allows us to select the best features and tune hyperparameters without biasing the model on the unseen test data set.

Definition

Hyperparameters: Hyperparameters are parameters that are set prior to the training process. They define the structure or behavior of the learning algorithm and are not learned from the data during training. Examples are the learning rate, activation function, or batch size.

Clarification

Features: Features is the data that is used as input for models to make predictions or decisions. They can be the attributes to describe each piece of data (e.g. the color, size, or shape of an object), or the combination or transformation of attributes.



4.9 K-fold Cross Validation

We partition the training data D_{train} into k subsets of equal size ($D_{train} = D_1 \cup D_2 \cup \dots \cup D_k$) as shown below.



Then we perform the below algorithm:

Definition

K-Fold algorithm: Given a choice of model (hyperparameters and features Φ) and $K \in \mathbb{N}$ (usually 5 or 10), do:

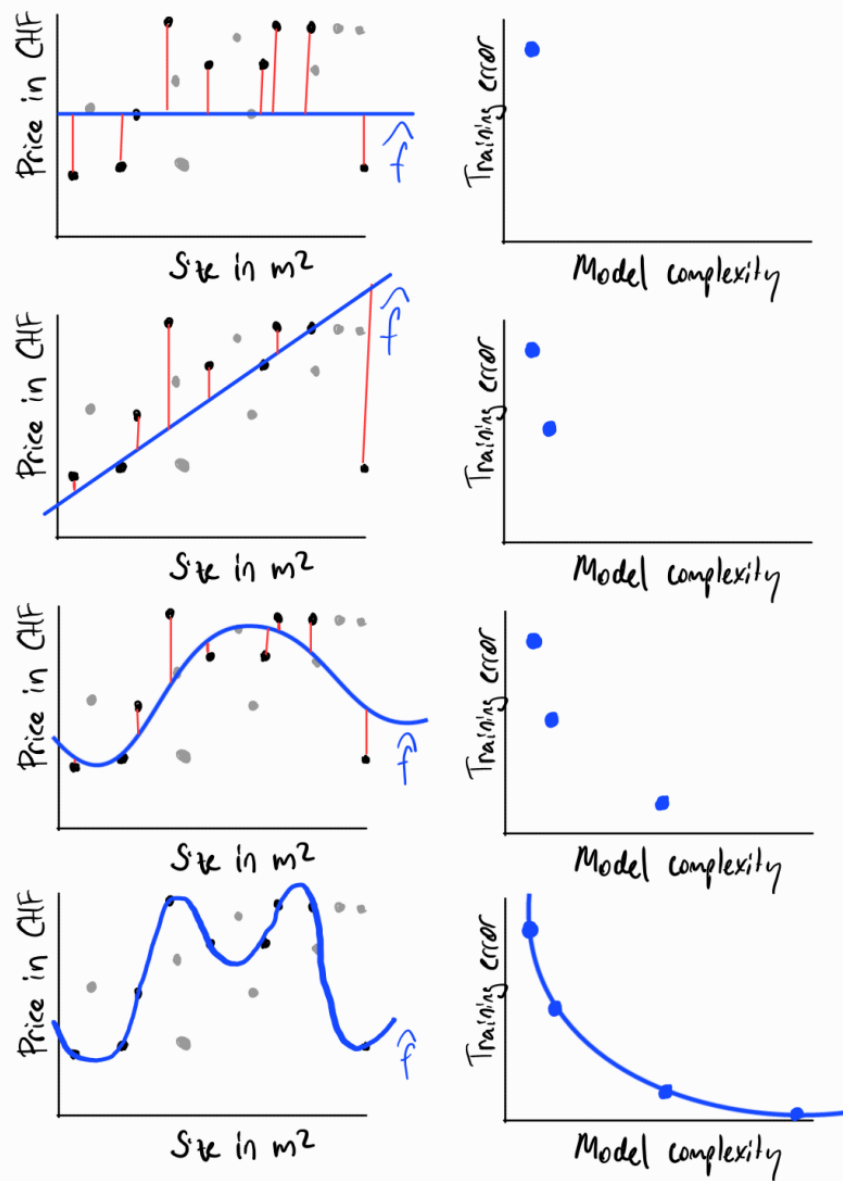
1. For all $k = 1, \dots, K$ (so for each partition):
 - (a) Compute $\hat{f}_k^\Phi$ that minimizes training loss on $D_{train} \setminus D_k$ using features Φ (notation: $\hat{f}_k^\Phi = M(D_k)$).
 - (b) Compute validation error on partition k : $L_k(\Phi) := \frac{1}{|D_k|} (\sum_{(x,y) \in D_k} l(\hat{f}_k^\Phi(x), y))$
2. Now we have K functions $\hat{f}_1^\Phi, \dots, \hat{f}_K^\Phi$ each trained on a different data set. Additionally, we have K validation errors $L_1(\Phi), \dots, L_K(\Phi)$. Now we compute the average validation error called "cross-validation error" $CV(\Phi) = \frac{1}{K} L_k(\Phi)$.

Note:

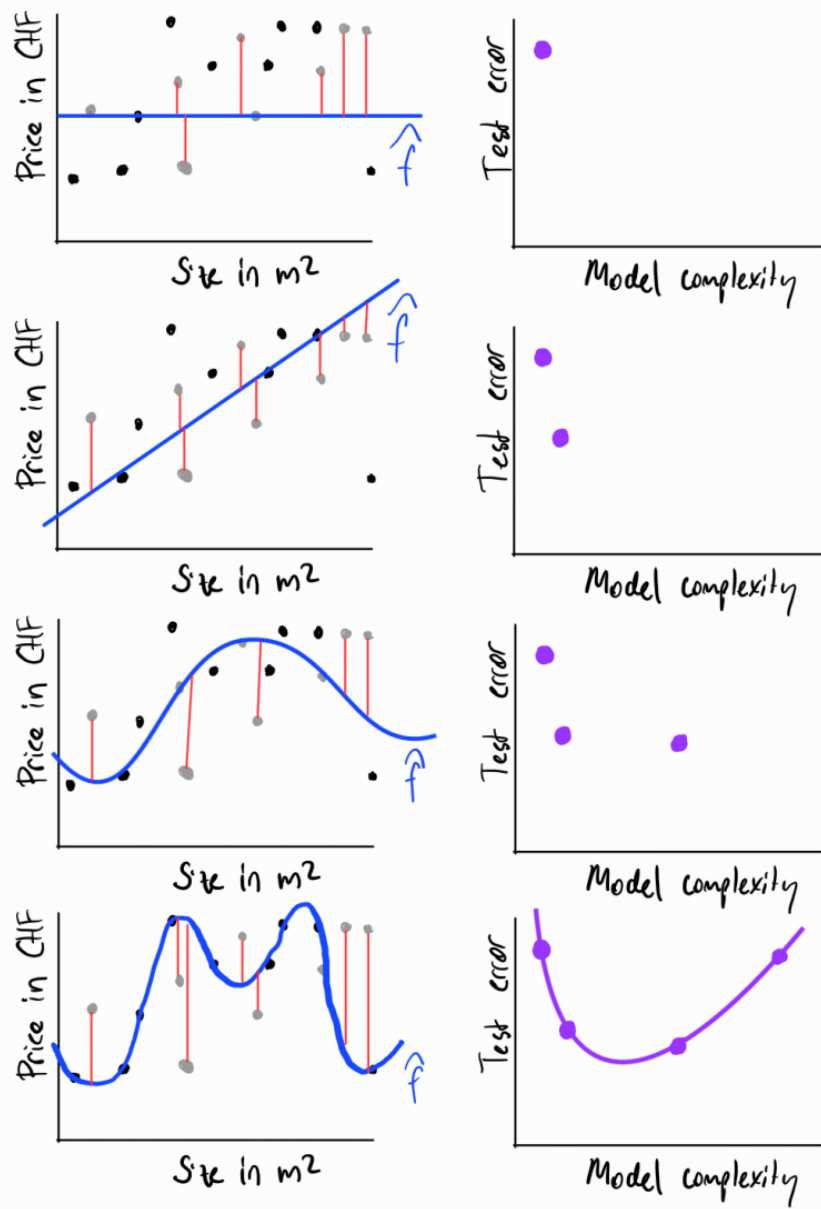
- For every choice of features and hyperparameters, we now have a corresponding cross-validation error $CV(\Phi)$
- We can now perform the above procedure for other choices of features and hyperparameters and choose the one with lowest cross-validation error
- Let the optimal features be Φ^* . We are now able to compute the final model $\hat{f}^{\Phi^*}$ on the entire training data D_{train}
- The generalization error of $\hat{f}^{\Phi^*}$ can be calculated using the held-out test data (generalization error = $L(\hat{f}^{\Phi^*}, D_{test})$).

4.10 Model Complexity

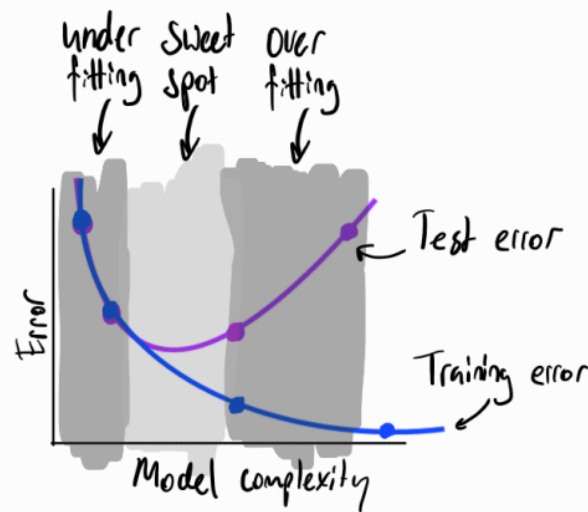
The training loss (loss that $\hat{f}$ generates on training data) decreases as the complexity of the model increases:



On the other hand, the generalization loss (loss that $\hat{f}$ generates on test data) is u-shaped:



We want to minimize the generalization loss:



Note that:

- Underfitting models predict...
 - ...training data **not well**
 - ...test data **not well**
- Right models predict...
 - ...training data **well**
 - ...test data **well**
- Overfitting models predict...
 - ...training data **well**
 - ...test data **not well**

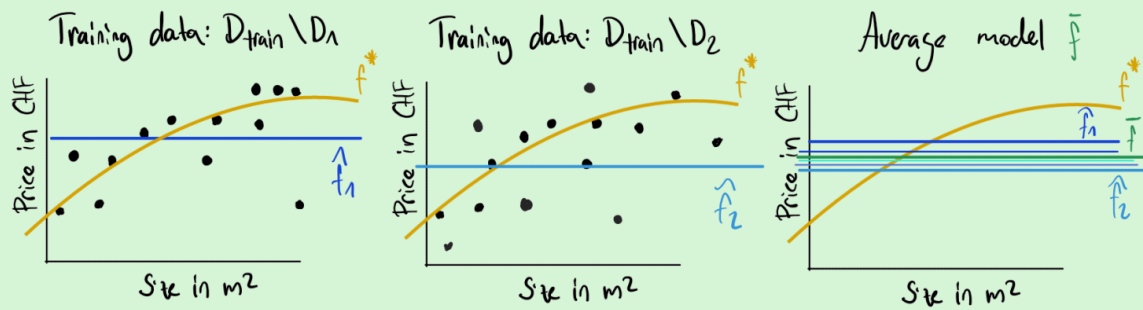
5 Bias-variance Tradeoff and Regularization

5.1 Bias-variance Tradeoff

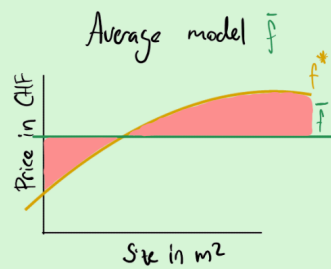
5.1.1 Bias

Definition

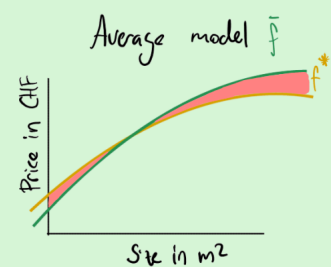
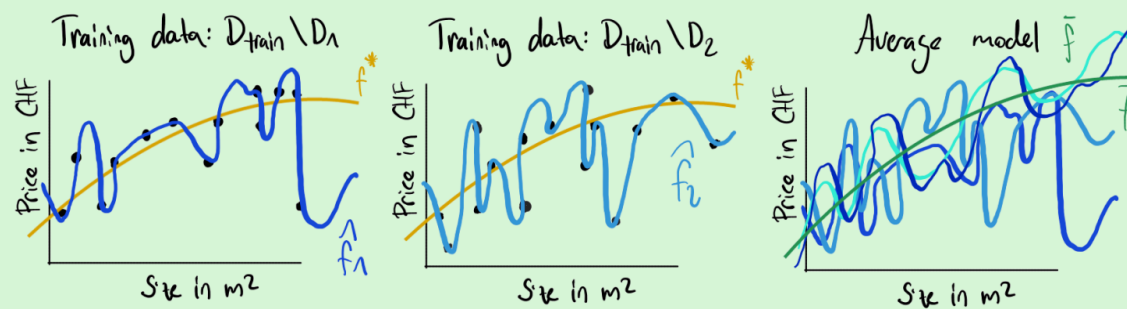
Bias: The bias of a model M is the expected squared distance of the average model $\bar{f} = \frac{1}{K} \sum_{i=1}^K \hat{f}_k^\Phi$ to the ground truth function f^* . It can be estimated as $\mathbb{E}_X(f^*(X) - \bar{f}(X))^2$ (though in practice we don't know f^*).



- Too simple models generally have high bias



- Too complex models generally have low bias

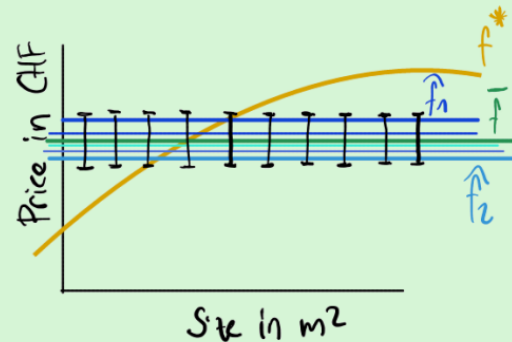


5.1.2 Variance

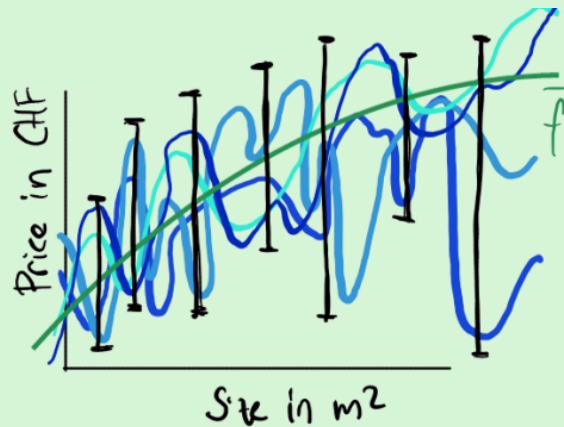
Definition

Variance: The variance of a model M is the expected squared distance of individual models $\hat{f}_k^\Phi = M(D_k)$ to the average model $\bar{f}$. It can be estimated as $\mathbb{E}_X(\frac{1}{K} \sum_{k=1}^K (f_k^\Phi(X) - \bar{f}(X))^2)$.

- Too simple models generally have small variance



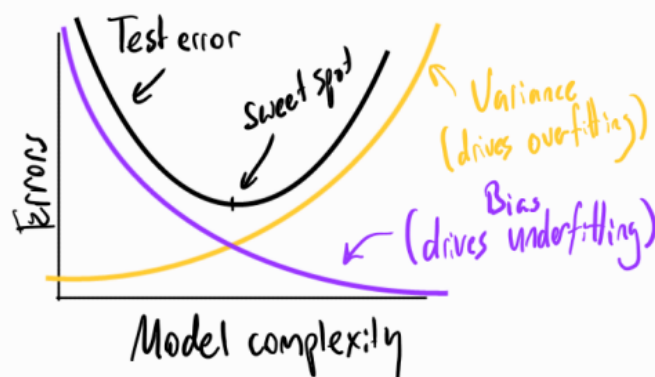
- Too complex models generally have high variance



5.1.3 Conclusion

Bias: Expected squared distance of average function to ground truth function

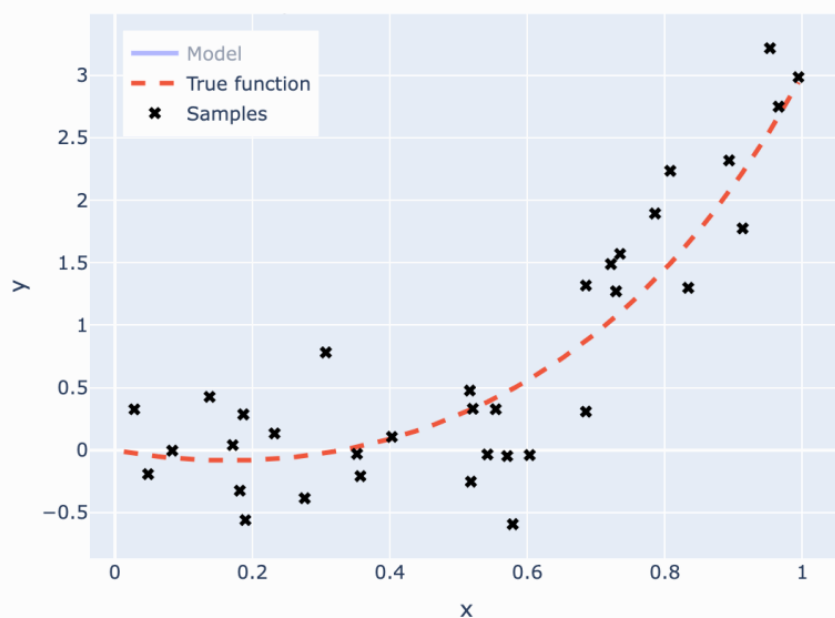
Variance: Expected squared distance of individual functions to average function



5.2 Regularization

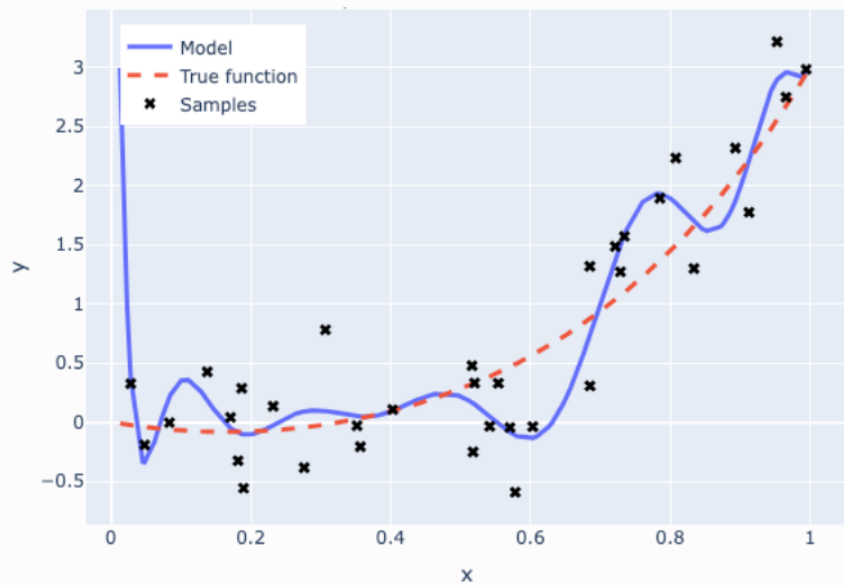
Regularization in machine learning is a technique used to prevent a model from overfitting on the training data. We know that overfitting occurs when a model learns the noise or random fluctuations in the training data instead of the actual underlying patterns, making it perform well on the training data but poorly on any unseen data. Regularization addresses this issue by adding a penalty on the size of the model parameters, encouraging the model to be simpler.

Example: Consider below ground truth function $f^* = -x + 3x^2 + x^5$ and $n = 35$ noisy samples $y = f^*(x) + \epsilon$ with $\epsilon \sim N(0, 0.4)$.



We choose the method: nonlinear regression with polynomial features $\phi(x) = (1, x, x^2, \dots, x^m)$ with degree m .

If we set $m = 12$ and use the minimizer $\operatorname{argmin}_{w \in \mathbb{R}^d} \|y - \Phi w\|_2^2$, then we get a polynomial of too high degree and experience overfitting.



We see that the polynomial function is "wiggly", which means that its coefficients are large.

Definition

1-Norm: $\|w\|_1 = \sum_{i=1}^d |w_i|$ (the sum of all absolute entries of the vector)

To make the polynomial function less wiggly, we can set a penalty on the coefficients:

$$\operatorname{argmin}_{w \in \mathbb{R}^d} \|y - \Phi w\|_2^2 \text{ such that } \|\hat{w}\|_1 \leq C (C \in \mathbb{N})$$

We just say that the final vector should not have too large coefficients.

Theorem

For any $C \in \mathbb{N}$ that we chose, there is a λ , such that:

$$\hat{w}_\lambda^{\text{lasso}} = \operatorname{argmin}_{w \in \mathbb{R}^d} \|y - \Phi w\|_2^2 + \lambda \|w\|_1$$

(we call this **Lasso regression**)

$$\hat{w}_\lambda^{\text{ridge}} = \operatorname{argmin}_{w \in \mathbb{R}^d} \|y - \Phi w\|_2^2 + \lambda \|w\|_2^2$$

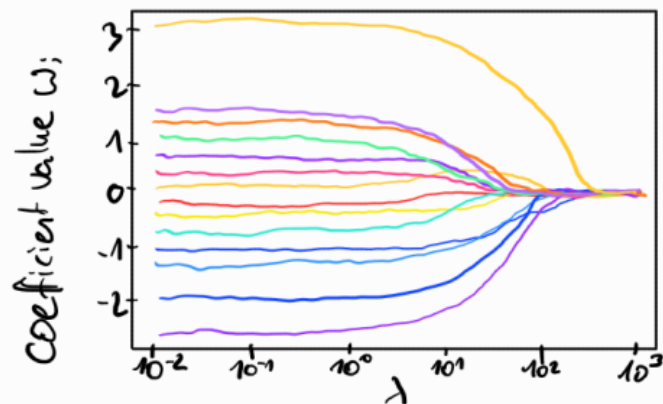
(we call this **ridge regression**). Note: we use squared 2-norm instead of 1-norm.

Clarification

The larger λ becomes, the smaller the 1-norm (in the case of lasso) or the squared 2-norm (in the case of ridge) are. This means that the polynomial function will be less wiggly and thus we get a simpler model.

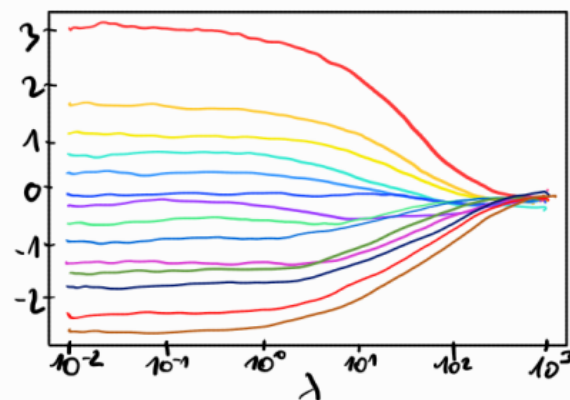
5.2.1 Comparing Lasso and Ridge Regression

Lasso:



As λ increases, one by one w_i become strictly zero
 $\rightarrow$ exact sparse solution

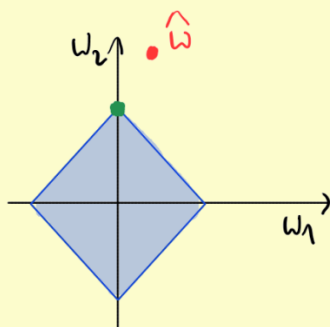
Ridge:



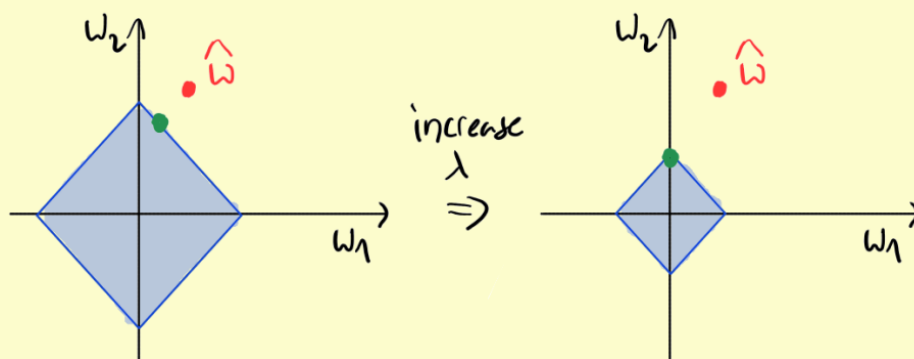
As λ increases, all w_i approach zero together
 $\rightarrow$ small Euclidean norm solution

Clarification

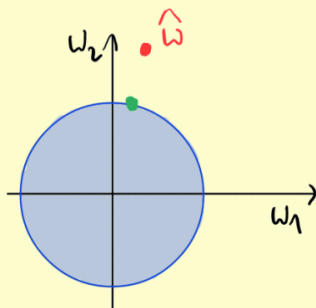
Lasso regression helps us find a sparser solution (i.e. a solution with coefficients are 0). This is because of the sharp diamond shape of $\|w\|_1 \leq C$. This makes it likely that the closest point to the optimum lies in one of the shape's sharp corners - and that is the point where some weights are 0.



It is always possible to set λ in such a way that a sparse solution is found.



Equivalently to the previous geometric intuition, **ridge regression** helps us find a denser solution.



We can use cross-validation to find an optimal λ .

1. Choose λ , Φ and $K \in \mathbb{N}$
2. For $k = 1, \dots, K$:
 - (a) Calculate $\hat{f}_k^\lambda = \operatorname{argmin}_f (L(f, D_{\text{train}} \setminus D_k) + \lambda \|f\|)$
 - (b) Calculate cross-validation loss on D_k : $CV_k^\lambda = \frac{1}{|D_k|} \sum_{(x,y) \in D_k} l(\hat{f}_k^\lambda(x), y)$
3. Compute average cross-validation loss among all folds: $CV^\lambda = \frac{1}{K} \sum_{i=1}^K CV_i^\lambda$
4. Repeat step 1 and 2 and pick the λ with lowest CV^λ
5. Train on entire training set and validate using the test set D_{test} .

6 Classification

6.1 Problem

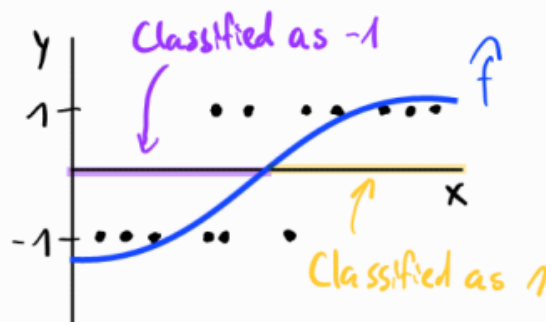
In regression we wanted to find a function that maps our inputs to some real value output ($\hat{f} : \mathbb{R}^d \rightarrow \mathbb{R}$). In classification we are interested in mapping our inputs to a finite, discrete set ($\hat{f} : \mathbb{R}^d \rightarrow \{1, 2, \dots, k\}$ for some $k \in \mathbb{N}$).

Examples are:

- **Binary classification:** Is this email spam (classes: spam / not spam)? Should we evacuate people considering the current weather conditions (classes: evacuation / no evacuation)?
- **Multiclass Classification:** Does this image show a cat, a dog, or a fish (classes: cat / dog / fish)? In which stage is this cancer (classes: 0, 1, 2, 3, 4)?

6.2 Binary Classification

1. We assign numerical values to both classes (e.g. healthy=-1, sick=1)
2. Similarly to regression, we can predict the class as $\hat{y} = \text{sign}(\hat{f}(x))$. So if $\hat{f}(x)$ is positive or zero, then we assume the classification to be 1. If $\hat{f}(x)$ is negative, then we assume the classification to be -1.
3. A good model predicts $\hat{y} = y$, where y is the true label



6.2.1 Exponential and Logistic loss

Our goal is to bring the generalization error (error on unseen data) down as much as possible.

$$\text{Generalization error of } \hat{f} : \mathbb{E}_{(x,y) \in D_{\text{test}}} [y \neq \text{sign}(\hat{f}(x))]$$

Definition

l_{0-1} : The pointwise loss function $l_{0-1}(f(x), y) = \begin{cases} 0 & \text{if } y = \text{sign}(f(x)) \\ 1 & \text{if } y \neq \text{sign}(f(x)) \end{cases}$

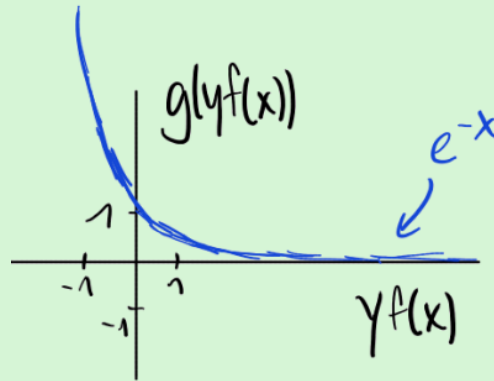
The idea in classification is to use different loss functions for training and testing. During testing we want the generalization to be as low as possible. And the generalization error is calculated using the l_{0-1} pointwise loss function.

But for training the model, we want to have a smooth, differentiable loss function which we can run gradient-based optimizations on. But for testing the model's performance, we use the exact l_{0-1} loss function.

But which training error do we use? One possibility is the exponential loss.

Definition

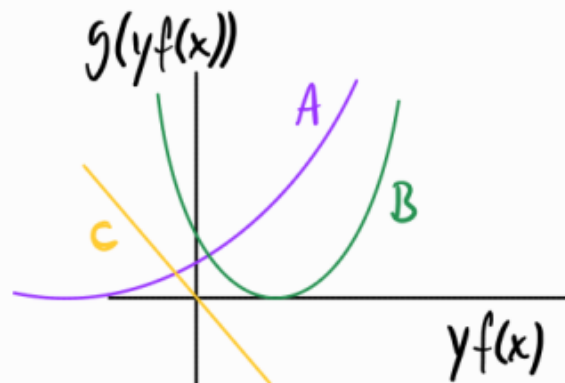
Exponential Loss: $l_{exp} = e^{-yf(x)}$.

**Clarification**

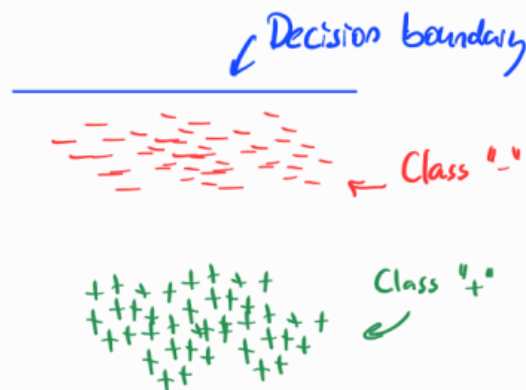
We want the sign of $y \cdot f(x)$ to be positive (when the sign of y and the sign of $f(x)$ are equal, then the sign of $y \cdot f(x)$ is positive). Therefore the loss function $g(yf(x))$ is small when $yf(x)$ is positive and large when $yf(x)$ is negative. The further right along the x-axis we go, the less the loss should become - because we want $yf(x)$ to become positive.

Additionally, the loss function should be differentiable so that it can be used in gradient descent. Strong convexity eliminates the problem of finding local minima or stationary points.

Other options are discussed below:



- **A:** This loss function points to the wrong direction. It incentivizes $yf(x)$ to be negative, which means that the signs of y and $f(x)$ are different. Therefore, this loss function is not suitable.
- **B:** This loss function works, however it is not optimal as it makes us optimize to a specific $yf(x)$ value (namely the minimum of the function).
- **C:** This loss function is negative if $yf(x) > 0$. This means that we get a reward the higher $yf(x)$ gets. This leads to problems as displayed below:



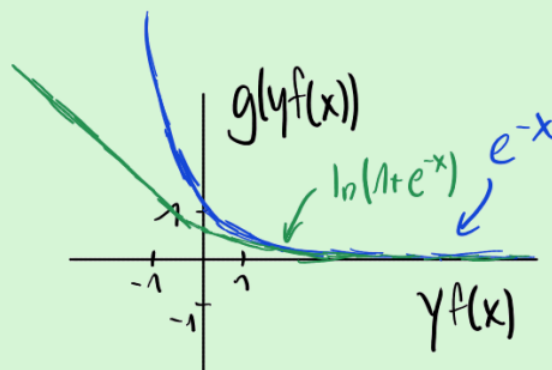
The distance to the green class of points is maximized with reward for linear loss, at expense of mistakes in the red class. This problem does not appear in the logistic loss, because it is strictly positive.

Apart from the exponential loss we discussed, there also is a logistic loss.

Definition

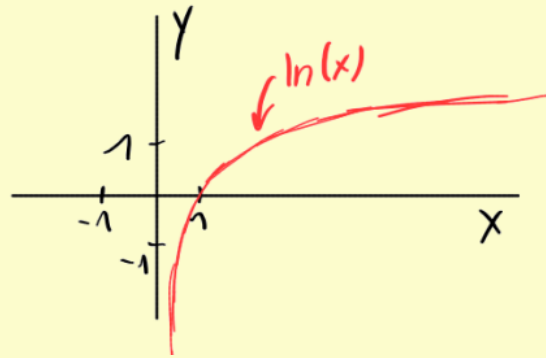
Logistic loss:

$$l_{\log}(f(x), y) = \log(1 + e^{-yf(x)})$$



Clarification

Why is there a +1 summand? If $yf(x)$ is a large positive number, then $-yf(x)$ is a large negative number. This means that $e^{-yf(x)}$ is a small positive number and $\log(e^{-yf(x)})$ is a large negative number. But this is a problem because the training function should be non-negative, which is why we add a +1 summand. Since $e^{-yf(x)}$ is a small positive number, $1 + e^{-yf(x)}$ is slightly larger than 1. This makes $\log(1 + e^{-yf(x)})$ positive.

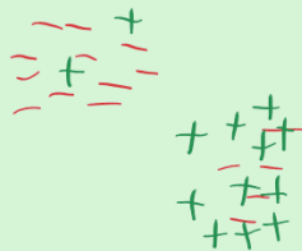


6.2.2 Maximum-Margin and Support Vector Machine

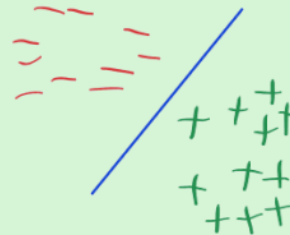
Definition

Linearly separable datasets: A dataset is considered linearly separable if you can draw a straight line (in two dimensions), a plane (in three dimensions), or a hyperplane (in more than three dimensions) to perfectly separate the different classes within the dataset. The below image illustrates this.

Not linearly separable:

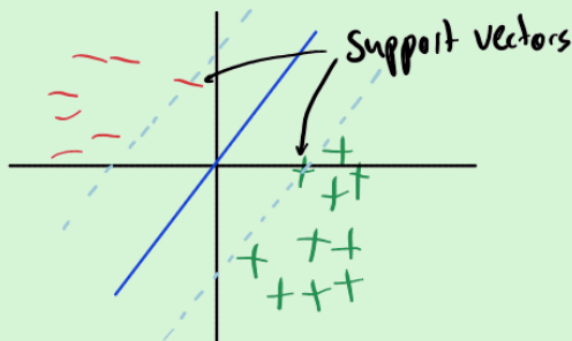


linearly separable:



Definition

Maximum-Margin Solution: The idea behind the maximum-margin solution is to find a decision boundary that separates the different classes in the dataset such that the distance between the decision boundary and the closest data point from each class is maximized. The closest points are called the support vectors, because they "support" the location and orientation of the decision boundary (i.e. the decision boundary is orthogonal to the support vector).



Why do we want to maximize the margin? A larger margin implies that the decision boundary is as far away as possible from any data point. This is desirable because it makes the classifier more robust to variations in the data (misclassification is less likely). It can be interpreted as the most "confident" separation between classes, which should lead to better performance on unseen data.

The maximum-margin solution is defined as $w_{mm} = \operatorname{argmax}_{\|w\|_2=1} (\min_i (y_i \langle w, x_i \rangle))$ (maximizes the minimum distance between the decision boundary and the closest point).

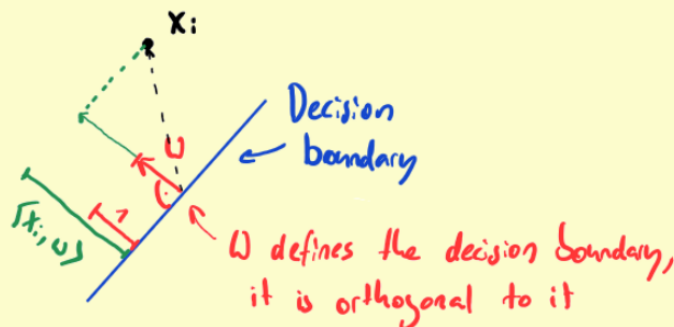
Clarification

Why is the distance between the decision boundary and the training sample (x_i, y_i) given by $y_i \langle w, x_i \rangle$?

- x_i are the coordinates of the point and y_i is the label (either 1 or -1)
- w is normalized and therefore has length 1 (See above $\operatorname{argmax}_{\|w\|_2=1}$)
- The dot product between two vectors $\langle a, b \rangle$ is defined as $a^T b = \sum_i a_i b_i$. There also is a geometric definition which is $\|a\| \|b\| \cos(\angle ab)$

Therefore,

$$\begin{aligned} \langle w, x_i \rangle &= \|w\| \|x_i\| \cos(\angle wx_i) && \text{geometric definition of } \langle \cdot, \cdot \rangle \\ &= \|x_i\| \cos(\angle wx_i) && \|w\| = 1 \\ &= \text{The length of vector } x_i \text{ projected in } w \text{ direction} \\ &= \text{The distance between the decision boundary and point } x_i \end{aligned}$$



This means that when $\langle w, x_i \rangle$ is positive, then the point (x_i, y_i) lies on the same side of the decision boundary where w points to. This is good if y_i is positive as well. But if y_i is negative, the point should be on the other side of the decision boundary. Therefore we multiply by y_i to make sure that each training sample x_i is correctly classified according to its label.

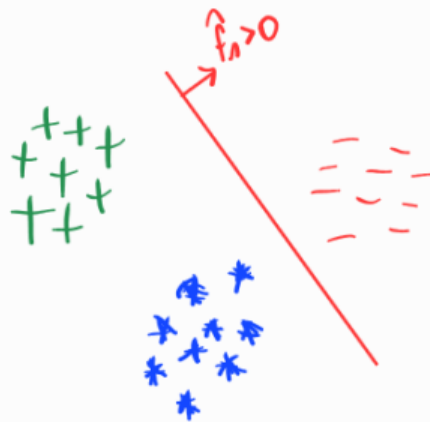
With $w_{mm} = \operatorname{argmax}_{\|w\|_2=1} (\min_i (y_i \langle w, x_i \rangle))$ we look for the normalized w vector that maximizes the distance between the decision boundary and the closest point of each class.

6.3 Multiclass classification

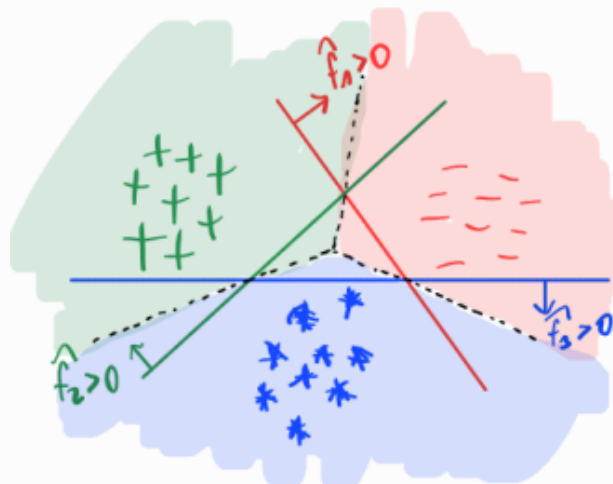
We might want to classify data into more than just two classes.

6.3.1 One-vs-Rest Approach

The one-vs-rest approach transforms the problem of multiclass classification into multiple binary classifications. For a classification problem with K classes, the one-vs-rest approach creates k separate binary classifiers ($\hat{f}_1, \dots, \hat{f}_k$). Each classifier is trained to distinguish between one of the classes and all other classes combined.



Then we can predict $\hat{y}(x) = \operatorname{argmax}_{i=1, \dots, k} \hat{f}_i(x)$.



But this means that we have to run k separate optimization procedures. Is there a way to train them simultaneously?

6.3.2 Softmax Function

The softmax function is a useful tool that allows us to perform this task. It converts a vector of raw scores (often called logits) into a **vector of probabilities** (all elements are between 0 and 1 and the sum over all elements is 1).

Definition

Softmax: Given a vector $a \in \mathbb{R}^n$ of size n , $\text{softmax}(a)$ is defined as:

$$\text{softmax}(a)_i = \frac{e^{a_i}}{\sum_{j=1}^n e^{a_j}}$$

Clarification

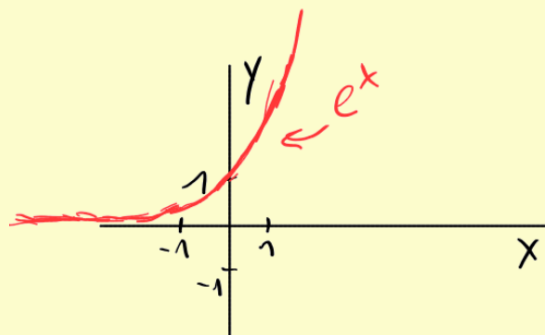
Example:

$$\text{softmax}\left(\begin{pmatrix} -1 \\ 1 \\ 2 \end{pmatrix}\right) = \begin{pmatrix} \frac{e^{-1}}{e^{-1}+e^1+e^2} \\ \frac{e^1}{e^{-1}+e^1+e^2} \\ \frac{e^2}{e^{-1}+e^1+e^2} \end{pmatrix} = \begin{pmatrix} 0.035119 \\ 0.259496 \\ 0.705385 \end{pmatrix}$$

- All elements are between 0 and 1
- All elements sum up to 1
- Element 2 is highest, and therefore most likely
- Element -1 is lowest, and therefore least likely

Softmax uses the exponential function because of its properties:

- $\forall x \in \mathbb{R} : e^x > 0$ (always positive - we don't want negative values in our output vector)
- $\forall x, y \in \mathbb{R} : x \geq y \implies e^x \geq e^y$ (monotonically increasing - higher values should lead to higher probabilities)



There also is a hardmax function, which prints 1 for the largest element in the input array and 0 for all other elements.

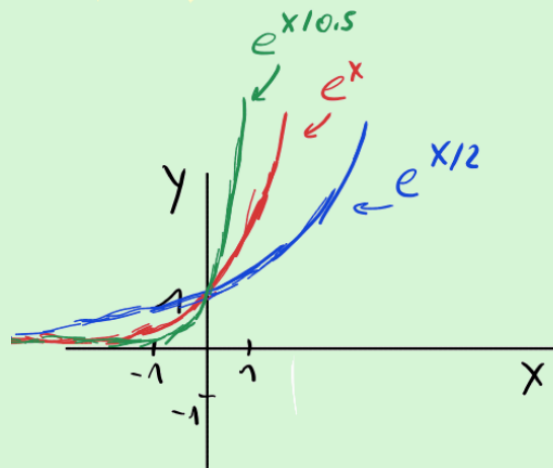
Definition

Temperature of softmax: We can introduce a constant T called temperature:

$$\text{softmax}(a)_i = \frac{e^{a_i/T}}{\sum_{j=1}^n e^{a_j/T}}$$

The temperature parameter scales the logits before applying the softmax function. This allows us to control the distribution of the probability output.

- **T=1:** The softmax function behaves normally, as the temperature does not scale the logits.
- **T>1:** The temperature increases, making the softmax output more uniform (i.e., the probabilities become more similar). This can be useful when we want the model to be less confident about its predictions or when we wish to encourage more exploration.
- **T<1:** The temperature decreases, making the softmax output more concentrated (i.e., increasing the disparity between the higher and lower probabilities). This can be helpful when we want the model to make more confident predictions. The softmax function behaves like the hardmax function as T converges towards 0.



6.3.3 Cross-entropy Loss

Our goal still is to train the k separate optimization procedures simultaneously. The most popular approach is to run gradient descent on the cross-entropy loss. Cross-entropy loss measures the performance of the classification model and uses softmax to do so.

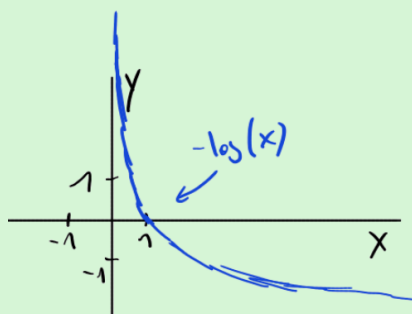
Definition

Cross-entropy loss: We feed the vector $f(x) := (\hat{f}_1(x), \dots, \hat{f}_k(x)) \in \mathbb{R}^k$ (which are the outputs of the different binary classifiers) into the cross-entropy loss:

$$l_{CE}(f(x), y) = -\log\left(\frac{e^{f_y(x)}}{\sum_{i=1}^k e^{f_i(x)}}\right)$$

Note:

- $f_y(x)$ is the output of the classifier $y \in \{1, \dots, k\}$ (which is also the label). This means we plug in the y th element of $\text{softmax}(f(x))$ into $-\log(\dots)$. Intuitively this means that we plug in the probability that the model is correct into $-\log(\dots)$.
- The reason we use $-\log(\dots)$ is because it satisfies the properties a loss function should have:



- It is non-negative for values between 0 and 1 (the y th element of $\text{softmax}(f(x))$ is a probability value between 0 and 1)
- $-\log(\dots)$ becomes monotonically smaller the higher the input value gets (which means that we get less loss if the probability value becomes larger)

6.3.4 Asymmetric Losses

We used the l_{0-1} loss (zero-one-loss) to determine how well our model performs on unseen data (the zero-one-loss prints 1 if our predicted classification is wrong, and 0 if it is correct). This loss might not be optimal in real-world applications.

Which covid diagnosis classifier should we use? We test with 32 samples.

		True label			
		sick	healthy	sick	healthy
Predicted label	sick	9	5	5	1
	healthy	1	5	5	9
		Classifier A		Classifier B	

If we want to prioritize public health, then we want to use classifier B. For prioritizing individual freedom we choose classifier A.

Instead of using the l_{0-1} loss that treats all errors symmetrically, we can weight errors differently by e.g. giving errors where $y = -1$ more weight (using constants c_{FN} for "False Negative" and c_{FP} for "False Positive").

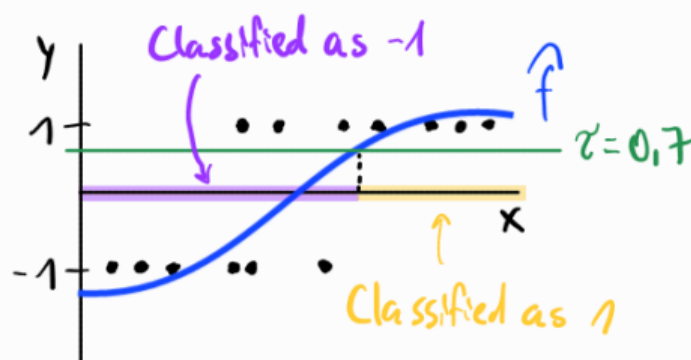
Truth

	$y = -1$	$y = 1$
Prediction	$\hat{y} = -1$	$\hat{y} = 1$
	TN	FN
	FP	TP

$$c_{FN} \frac{1}{\#y = 1} \sum_{(x,y):y=1} 1_{\hat{y}=-1} + c_{FP} \frac{1}{\#y = -1} \sum_{(x,y):y=-1} 1_{\hat{y}=1} = c_{FN} \frac{\#FN}{\#y = 1} + c_{FP} \frac{\#FP}{\#y = -1}$$

Given a $\hat{f}$, we can obtain smaller loss $c_{FN} \frac{\#FN}{\#y=1} + c_{FP} \frac{\#FP}{\#y=-1}$ by changing τ . τ is the threshold value that we use to decide if our prediction y is 1 or -1 . We used to have $\tau = 0$:

$$\hat{y}_\tau = \text{sign}(\hat{f}(x)) = \begin{cases} +1 & \text{if } \hat{f} \geq 0 \\ -1 & \text{if } \hat{f} < 0 \end{cases}$$

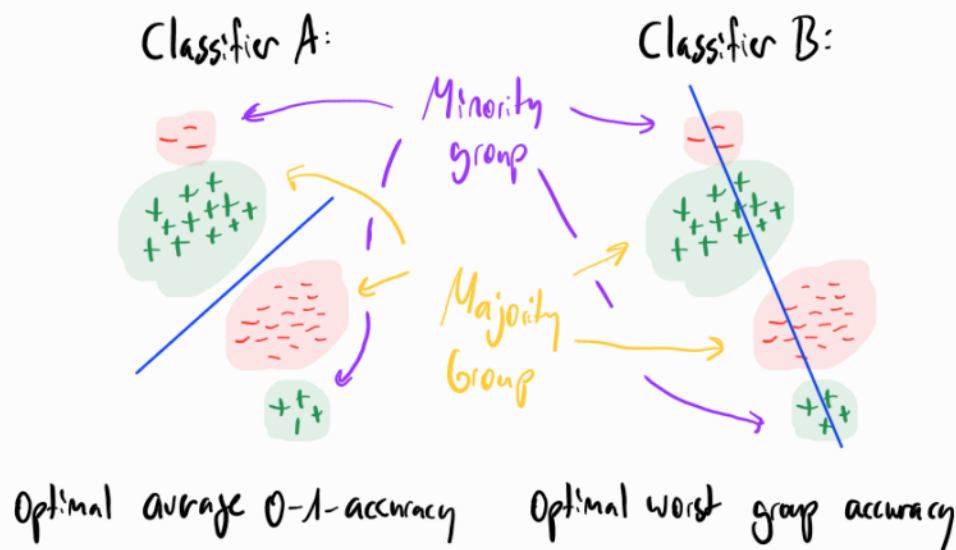


6.3.5 Receiver Operating Characteristic (ROC)

6.4 Trustworthy Generalization

Consider the previous example where we wanted to classify cats, dogs and ships. Often times these classes can be split further into subgroups (e.g. male/female cats or dogs, albino animals, etc.).

It might be that a classifier has low error in classifying general input data correctly, but has high error on specific subgroups.



- **Classifier A:** Low overall error, but high error on minority groups
- **Classifier B:** Difference in error among subgroups is smaller, but overall error is higher

There are several possible notions of "fair" classifiers:

- **Equalized odds:** all groups have similar error rate ($\mathbb{P}(\hat{Y} = 1|Y = y, S = g)$ equal for all $g \in G, y \in \{0, 1\}$ where $\hat{Y}$ is the predicted value)
- **Demographic parity:** Across group equal percentage predicted as 1 vs -1 ($\mathbb{P}(\hat{Y} = 1|S = g)$ equal for all $g \in G$)
- **Equalized opportunity:** All groups should have similar "false negative rate" ($\mathbb{P}(\hat{Y} = 1|Y = 1, S = g)$ equal for all $g \in G$)
- **Worst-group error:** The largest error among all groups is small ($\max_{g \in G} \sum_y \mathbb{P}(\hat{Y} = -y|Y = y, S = g)$ small)
- **Balanced error:** Average of all group errors is small ($\frac{1}{|G|} \sum_g \sum_y \mathbb{P}(\hat{Y} = -y|Y = y, S = g)$)

7 Kernel Methods

7.1 Improving Polynomial Regression

7.1.1 Computational Complexity

How large must p be (the dimension of the below features vector) to express a degree m polynomial for input data $x \in \mathbb{R}^d$?

$$\Phi(x) = [1, x_{[1]}, \dots, x_{[d]}, x_{[1]}^2, \dots, x_{[d]}^2, \dots, x_{[1]}x_{[2]}, \dots, x_{[1]}x_{[d]}, \dots, x_{[1]}^2x_{[2]}, \dots] \in \mathbb{R}^p$$

A polynomial of degree m is a linear combination of monomials of degree smaller or equal to m , where the coefficient of at least one monomial with degree m is non-zero (monomial of degree m can be $x_{[1]}^m$ but also combinations of different attributes such as $x_{[1]}^{m-1}x_{[2]}$)

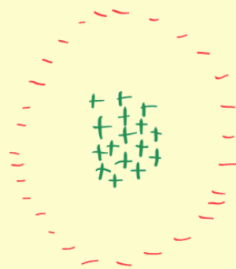
Therefore, all monomials with degree smaller or equal to m must be present in $\Phi(x)$ (e.g. $1, x_{[1]}, \dots, x_{[1]}^m, \dots$ but also $x_{[1]}^{m-2}x_{[2]}^2 \dots$). This means that p (the size of $\Phi(x)$) grows exponentially as larger d and m become. The key message is that we quickly obtain a "feature explosion" with large memory and computational requirements, especially when working with high dimensional data.

7.1.2 Kernel trick

To avoid operating in this huge feature space, there is something called the kernel trick. The kernel trick helps us to efficiently find non-linear decision boundaries (e.g. polynomial) without explicitly computing the coordinates of the data in the feature space.

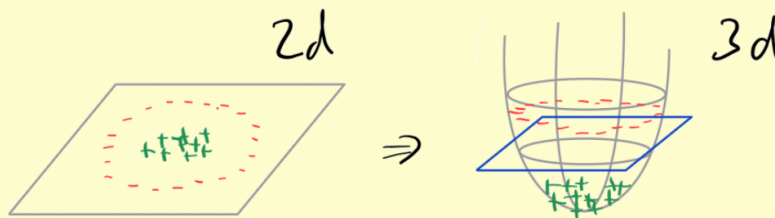
Clarification

Example: Suppose we have the below data points.

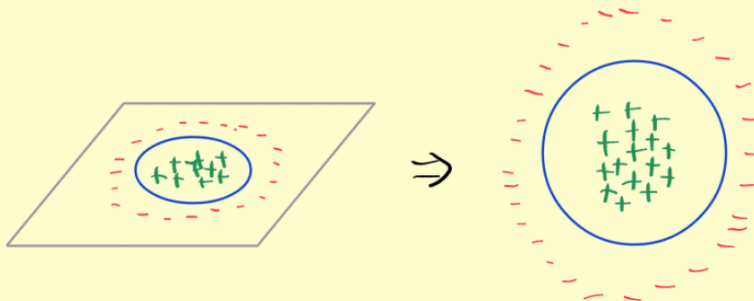


It is obvious that the dataset is not linearly separable. Therefore we need to find a non-linear decision boundary.

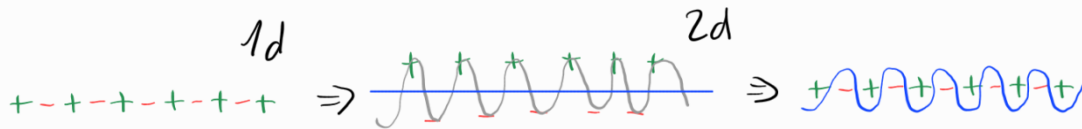
The idea is to transform the dataset into a **higher dimension**, in order to make it linearly separable. As displayed below, we transformed the dataset into higher dimension - now it is linearly separable and we can efficiently find a decision boundary.



Now we can reverse the transformation to make the dataset return to its original dimension.



There are many possible functions that can map the data to any number of higher dimensions such that it becomes linearly separable.



There can be many transformations that allow the data to be linearly separated in higher dimensions, but not all of these functions are actually "kernels" (defined soon).

We saw how higher dimensional transformations can allow us to separate data in order to make easy classifications. It seems that for finding the classifiers, we would have to perform operations with the higher dimensional vectors in the transformed feature space ($\Phi(x)$). But we don't want to work with these higher dimensional data because of additional computational cost.

The **kernel trick** provides a solution to this problem. The "trick" is that kernel methods represent the data only through a set of pairwise "similarity comparisons" (a.k.a. scalar product) in the original lower dimensional space, instead of explicitly calculating the transformations $\Phi(x)$.

Definition

Kernel: A function that takes vectors in the original space as its inputs and returns the dot product of the vectors in the feature space.

$$k(a, b) = \langle \Phi(a), \Phi(b) \rangle$$

So the kernel function is some arbitrary function that accepts inputs in the original lower dimensional space and returns the same result as the dot product of the transformed vectors in the higher dimensional space, without the overhead of having to compute the coordinates of the vectors in the high-dimensional space.

The trick is to use such a kernel function instead of calculating $\Phi(a)$ and $\Phi(b)$.

Clarification

Example: For the feature space $\Phi(x) = (1, \sqrt{2}x_1, \sqrt{2}x_2, \sqrt{2}x_1x_2, x_1^2, x_2^2)$ (feature space of 2nd degree polynomials in $x = (x_1, x_2)^T$), $k(a, b) = (1 + a^T b)^2$ is a valid kernel ($a, b \in \mathbb{R}^2$).

$$\begin{aligned}
 k(a, b) &= (1 + a^T b)^2 && / \text{ def of } k \\
 &= (1 + a_1b_1 + a_2b_2)^2 && / \text{ dot product} \\
 &= (1 + 2a_1b_1 + 2a_2b_2 + 2a_1b_1a_2b_2 + a_1^2b_1^2 + a_2^2b_2^2) && / \text{ expand} \\
 &= \langle (1, \sqrt{2}a_1, \sqrt{2}a_2, \sqrt{2}a_1a_2, a_1^2, a_2^2), (1, \sqrt{2}b_1, \sqrt{2}b_2, \sqrt{2}b_1b_2, b_1^2, b_2^2) \rangle && / \text{ rewrite} \\
 &= \langle \Phi(a), \Phi(b) \rangle && / \text{ def of } \Phi
 \end{aligned}$$

This means that instead of computing $\Phi(a)$ and $\Phi(b)$ (which is memory intensive because we bring the data into higher dimensional space), we can simply calculate $(1 + a^T b)^2$ to get the dot product of the Φ -vectors.

To calculate the training loss, we don't need the actual value of $\Phi(x)$ (where x is some data sample), but we need their dot product $\langle \Phi(a), \Phi(b) \rangle$ (a and b are some data samples).

Theorem

Mercer's theorem: A function must satisfy the below two properties to be a kernel:

- Symmetry: $k(a, b) = k(b, a)$. Symmetry must hold because

$$\begin{aligned}
 k(a, b) &= \langle \Phi(a), \Phi(b) \rangle && / \text{ def of } k \\
 &= \langle \Phi(b), \Phi(a) \rangle && / \text{ dot product is symmetric} \\
 &= k(b, a) && / \text{ def of } k
 \end{aligned}$$

- Positive semi-definiteness: The kernel matrix $K = \begin{pmatrix} k(x_1, x_1) & \cdots & k(x_1, x_n) \\ \vdots & \ddots & \vdots \\ k(x_n, x_1) & \cdots & k(x_n, x_n) \end{pmatrix}$ is positive-

semidefinite for any choice of inputs $x_1, \dots, x_n$. This means that $\forall c \in \mathbb{R}^n : c^T K c \geq 0$.

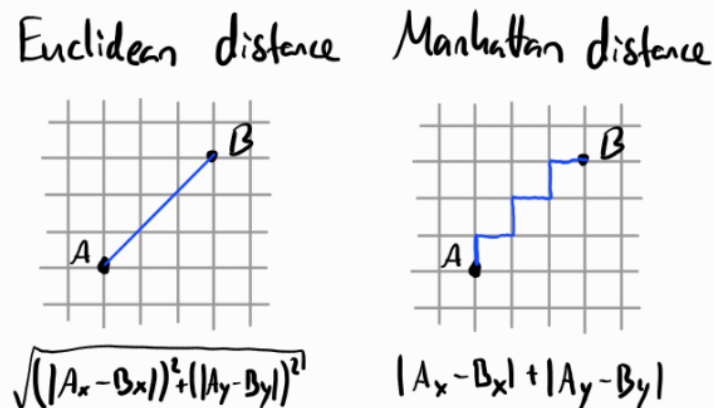
Positive semi-definiteness can geometrically be interpreted as follows: We take any vector c and transform it using the linear map K . Then we take the dot product between the transformed vector Kc and the original vector c to see if Kc still "points in the same direction as c " (meaning that the angle between Kc and c is smaller than or equal to 90°). If this is the case for all c , then we call K positive-semidefinite.

7.2 Other nonlinear methods

7.2.1 k -Nearest Neighbor

The idea behind the k -NN approach is to make predictions for new data points based on the majority class (in classification) or the average (in regression) of their k nearest neighbors in the training data.

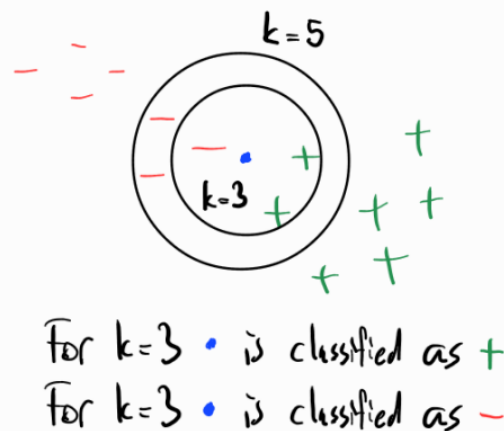
For a new data point x that we want to classify or predict its value, the algorithm finds the k closest data points (neighbors) to x in the training data based on some distance metric (e.g. Euclidean distance or Manhattan distance).



For classification, the algorithm assigns the most frequent class among the k nearest neighbors to the new data point x . For regression, the algorithm calculates the average (or weighted average) of the target values of the k nearest neighbors and assigns this value to x .

This means, that there is no explicit training performed for k -NN.

The parameter k (the number of neighbors to consider) is a hyperparameter of the algorithm, and the optimal k can be found using cross-validation. k needs to be chosen carefully because it significantly influences the model's behavior. The choice of distance metric (e.g. Euclidean, Manhattan, or others) also affects the algorithm's performance. Different distance metrics might be more suitable for different types of data and problems.



*Correction: for $k = 5$, • is classified as -.

k -NN is...

- + ...simple to understand and implement
- + ...does not require a training phase
- + ...makes no assumptions about the underlying distribution of the data, which means it can be effective for datasets where the distribution of the test samples is unknown
- - ...computationally expensive, because the algorithm needs to compute distances to all training samples, which can be slow and memory-intensive for large datasets
- - ...sensitive to noise and outliers in the data, because noisy data or outliers can affect the predictions, especially when using small k
- - ...As the number of features (dimensions) increases, the distance between data points becomes less meaningful, leading to decreased performance (curse of dimensionality).

Clarification

Example: Imagine the below data:

$$\begin{pmatrix} 0.3 & 0.5 & NaN & 0.2 & 0.7 \\ 0.4 & 0.6 & 0.2 & 0.3 & 0.8 \\ 0.2 & 0.4 & 0.1 & 0.3 & 0.6 \\ 0.1 & 0.3 & 0.1 & 0.2 & 0.5 \\ 0.7 & 0.9 & 0.5 & 0.8 & 0.9 \end{pmatrix}$$

It has missing data in the first row. We can use the k -NN approach to predict it.

1. Calculate the distance to every other data point (e.g. using Euclidean distance, $k = 3$), ignoring the missing dimension.
 - **distance to 2nd row:** $\sqrt{(0.3 - 0.4)^2 + (0.5 - 0.6)^2 + (0.2 - 0.3)^2 + (0.7 - 0.8)^2} = \sqrt{0.01 + 0.01 + 0.01 + 0.01} = \sqrt{0.04} = 0.2$
 - **distance to 3rd row:** $\sqrt{(0.3 - 0.2)^2 + (0.5 - 0.4)^2 + (0.2 - 0.3)^2 + (0.7 - 0.6)^2} = \sqrt{0.01 + 0.01 + 0.01 + 0.01} = \sqrt{0.04} = 0.2$
 - **distance to 4th row:** $\sqrt{(0.3 - 0.1)^2 + (0.5 - 0.3)^2 + (0.2 - 0.2)^2 + (0.7 - 0.5)^2} = \sqrt{0.04 + 0.04 + 0 + 0.04} = \sqrt{0.12} = 0.346$

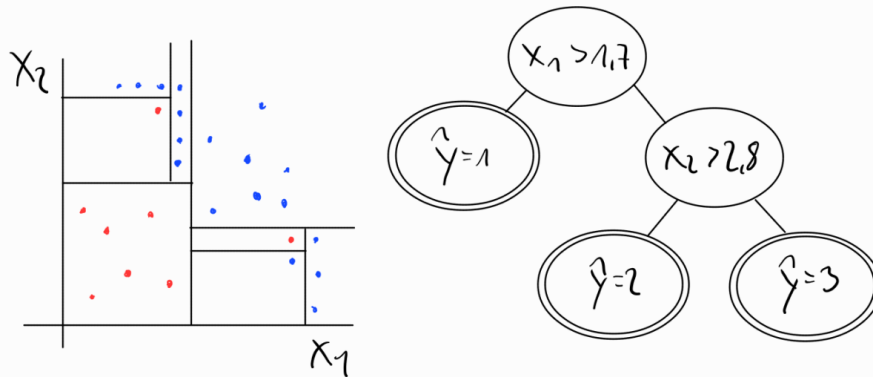
- **distance to 5th row:** $\sqrt{(0.3 - 0.7)^2 + (0.5 - 0.9)^2 + (0.2 - 0.8)^2 + (0.7 - 0.9)^2} = \sqrt{0.16 + 0.16 + 0.36 + 0.04} = \sqrt{0.72} = 0.848$

2. Identify k nearest neighbors: rows 2, 3, 4.

3. Predict missing value by calculating the average of the k nearest neighbors: $\frac{0.2+0.1+0.1}{3} = 0.1333$.

7.2.2 Decision Trees

The idea behind decision trees is to learn a series of hierarchical if-else decision rules to predict the value of a new data point. By using these if-else decision rules we partition the data as shown below.



Decision trees are...

- + ...intuitive and relatively easy to interpret
- + ...makes no assumptions about the underlying distribution of the data, which means it can be effective for datasets where the distribution of the test samples is unknown
- - ...are prone to overfitting, especially when the tree depth is large (many if-else decisions) and the dataset is noisy
- - ...usually relying on greedy methods to find the decision rules, which can be suboptimal as bad decision choices on the top nodes propagate to the leaves

8 Neural Networks

8.1 Learning Features

So far, the features Φ were fixed during training. In neural networks, the features are not fixed but trained. To come up with the best possible features might be hard. For instance: What are the best features for classifying the below written digits?



Is it pixels? Groups of pixels? Edges? Others?

Hand-designing features requires domain knowledge, which is not always given. The idea is to parameterize the features as well.

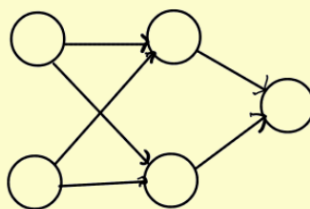
Clarification

Example: It's best to explain using a practical example. We want to predict whether an email is spam or not.

- **Inputs:**

1. real number between 0 and 1 indicating the ratio of keywords related to spam emails (e.g. "prince", "free", "winner", etc.).
2. discrete number 0 or 1 indicating whether the email was sent in out-of-office hours (e.g. 1 if email was sent between 6 pm and 7 am).

- **Input layer:** 2 nodes called a_1 and a_2 (one node per feature)
- **Hidden layer:** 1 hidden layer with 2 nodes
- **Output layer:** 1 node indicating the likelihood of email being spam



Below are the initial weights:

- **First layer to second layer:**

$$w^{(1)} = \begin{pmatrix} 0.25 & 0.25 \\ -0.25 & -0.25 \end{pmatrix}$$

- **Second layer to third layer:** $w^{(2)} = (0.5 \quad 0.5)$

Biases:

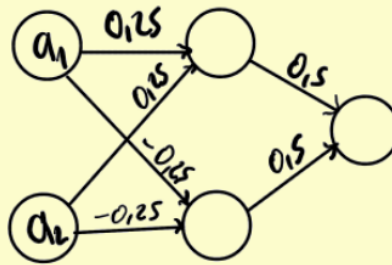
- **First layer to second layer:**

$$b^{(1)} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

- **Second layer to third layer:** $b^{(2)} = (0)$

Note that we have $|E|$ weights (amounts of edges in the neural network graph) and $|V| - |V_{\text{Input layer}}|$ biases (amounts of nodes without input layer).

Forward Pass:

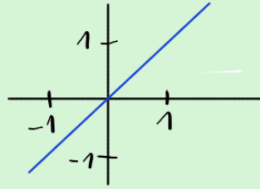


Calculating the activation of the first neuron in the hidden layer, considering input "input 1: $a_1 = 0.1$, input 2: $a_2 = 1$ ": $(a_1 \cdot w_{11}^{(1)} + a_2 \cdot w_{12}^{(1)} + b_1^1) = (0.1 \cdot 0.25 + 1 \cdot 0.25 + 0) = 0.275$. We now want to normalize the output using an activation function - for instance to map the activation to a value between 0 and 1.

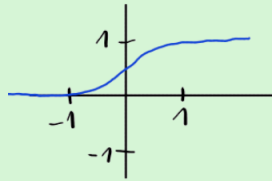
Definition

Examples of activation functions:

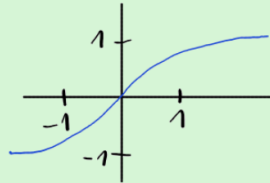
- **Identity:** $\sigma_{id}(x) = x$



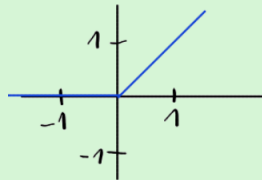
- **Sigmoid:** $\sigma_{sig}(x) = \frac{1}{1+e^{-x}}$



- **Tanh:** $\sigma_{tanh}(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$



- **Rectified Linear Unit (ReLU):** $\sigma_{ReLU}(x) = \max(0, x)$



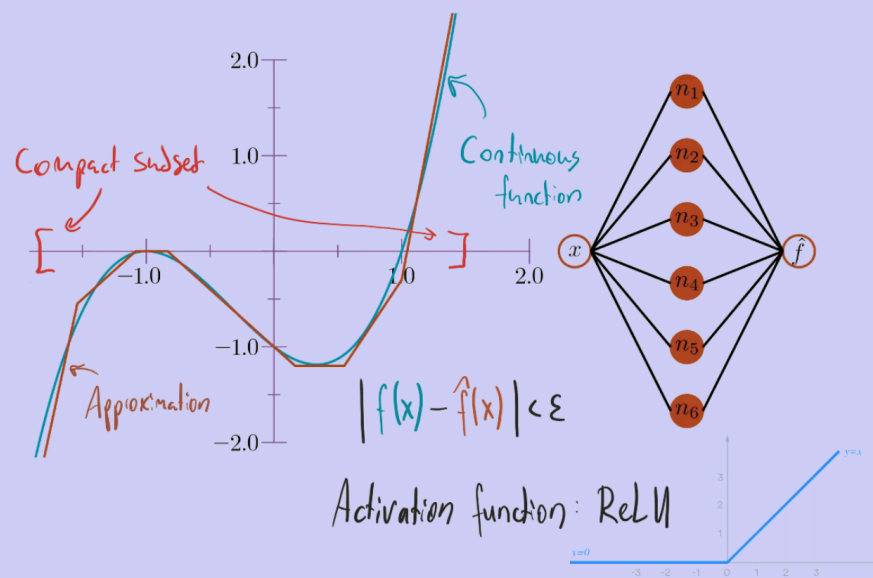
If we use the sigmoid activation function, then we get $\sigma_{sig}(0.275) = 0.56832$.

To calculate the activations of the first layer, we can also use matrix-vector-notation:

$$\begin{aligned}\sigma_{sig}(w^{(1)}a + b^{(1)}) &= \sigma_{sig}\left(\begin{pmatrix} 0.25 & 0.25 \\ -0.25 & -0.25 \end{pmatrix} \begin{pmatrix} 0.1 \\ 1 \end{pmatrix} + \begin{pmatrix} 0 \\ 0 \end{pmatrix}\right) \\ &= \sigma_{sig}\left(\begin{pmatrix} 0.275 \\ -0.275 \end{pmatrix}\right) \\ &= \begin{pmatrix} 0.56832 \\ 0.43168 \end{pmatrix}\end{aligned}$$

Theorem

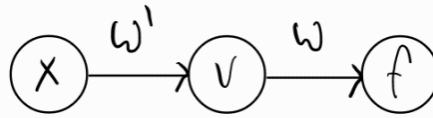
Universal Approximation Theorem: The Universal Approximation Theorem states that a neural network with at least one hidden layer, using a **non-linear activation function**, can approximate any **continuous** function within a **compact set** in $\mathbb{R}^n$ to any desired degree of accuracy, given sufficient number of neurons in the hidden layer.



Several questions arise:

- Which features should be used?
- How many hidden layers should be used?
- How should the weights be initialized?
- What activation function should be used?

We try to answer these questions in the next chapters. First, let's consider another simple example to understand "backwards pass".



- x is the only node in the input layer (feature)
- v is the only node in the hidden layer
- f is the only node in the output layer

Forward pass: v is calculated as $v = \sigma(w' \cdot x)$, and f as $f = w \cdot v$ (in this example we don't use an activation function in the last layer). Furthermore, we don't use any biases.

This means our neural network can be described as $f(W; x) = f([w'], (w); x) = w\sigma(w'x)$.

Backward pass: The squared loss function is shown below (se use $\frac{1}{2}$ for simplification - it doesn't change the location of the minimum of the function):

$$\begin{aligned} l(W; x, y) &= \frac{1}{2}(y - f(x))^2 && / \text{ def square loss} \\ &= \frac{1}{2}(y - w\sigma(w'x)) && / \text{ def } f \end{aligned}$$

Next we calculate the derivative with respect to the weights W .

With respect to w :

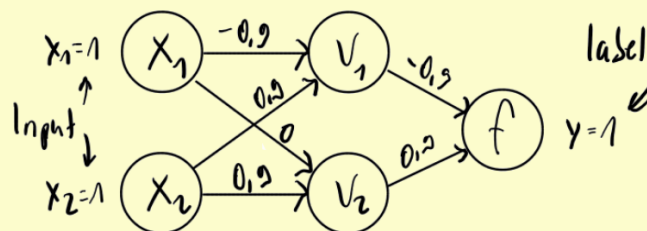
$$\begin{aligned} \frac{dl}{dw} &= \frac{dl}{df} \frac{df}{dw} && / \text{ chain rule} \\ &= (y - f)(-1)\sigma(w'x) \\ &= (f - y)v && / \text{ def } v \end{aligned}$$

With respect to w' :

$$\begin{aligned} \frac{dl}{dw'} &= \frac{dl}{df} \frac{df}{dw'} && / \text{ chain rule} \\ &= \frac{dl}{df} \frac{df}{dv} \frac{dv}{dz} \frac{dz}{dw'} && / \text{ chain rule, } z = w'x \\ &= (f - y)w\sigma'(w'x)x && / \sigma' \text{ is the derivative of } \sigma \end{aligned}$$

Clarification

Example: Consider the below neural network.



Forward pass: Assume:

- Inputs $x_1 = 1, x_2 = 1$
- ReLU activation function
- Loss function: $l = (y - f)^2$ (squared loss)

This leads to:

- $v_1 = 1 \cdot -0.9 + 1 \cdot 0.9 = 0$
- $v_2 = 1 \cdot 0 + 1 \cdot 0.9 = 0.9$
- $f = 0 \cdot -0.9 + 0.9 \cdot 0.9 = 0.81$

Let $W^{(2)}$ be the weights between the second layer and the output layer ($W^{(2)} = \begin{pmatrix} -0.9 & 0.9 \end{pmatrix}$). To perform backward propagation with these weights we calculate:

$$\begin{aligned}
 \frac{dl}{dW^{(2)}} &= \frac{dl}{df} \frac{df}{dW^{(2)}} \\
 &= \frac{dl}{df} \frac{dW^{(2)}v}{dW^{(2)}} && / f = W^{(2)}v \text{ matrix multiplication} \\
 &= \frac{dl}{df} v^T && / \frac{dW^{(2)}v}{dW^{(2)}} = v^T \\
 &= \frac{d(y-f)^2}{df} \begin{pmatrix} 0 & 0.9 \end{pmatrix} && / \text{def of } l \text{ and } v^T \\
 &= 2(1 - 0.81)(-1) \begin{pmatrix} 0 & 0.9 \end{pmatrix} && / \text{label } y = 1 \\
 &= \begin{pmatrix} 0 & -0.342 \end{pmatrix}
 \end{aligned}$$

This is the gradient of the loss function.

Next we perform gradient descent to optimize the weights $W^{(2)}$.

$$\begin{aligned}
 W_{new}^{(2)} &= W^{(2)} - \eta \frac{dl}{dW^{(2)}} && / \text{def gradient descent} \\
 &= \begin{pmatrix} -0.9 & 0.9 \end{pmatrix} - 0.2 \begin{pmatrix} 0 & -0.342 \end{pmatrix} && / \text{step size } \eta = 0.2 \\
 &= \begin{pmatrix} -0.9 & 0.9684 \end{pmatrix}
 \end{aligned}$$

With these new weights the neural network is able to predict the true label 1 better than before.

9 Clustering

So far we looked at supervised learning. The idea behind unsupervised learning is that the dataset does not contain any labels, and let the model learn the labels itself based on some goals (e.g. grouping the data by similarities, understanding meaningful features, etc.).

As the name suggests, the idea behind clustering is that we group data points into clusters such that similar data points are assigned to the same cluster. Sometimes it is also the idea to identify data points that don't fit in any cluster (outliers).

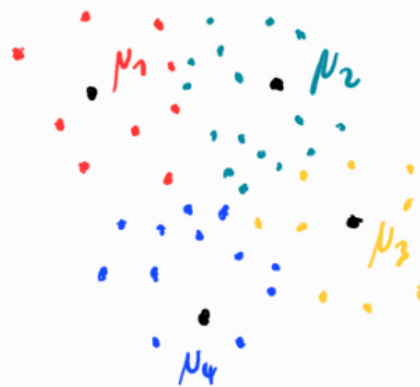
- **Hierarchical clustering:** The idea of hierarchical clustering is to seek a hierarchy of clusters. Hierarchical clustering is useful when the relationship among the data points is more important than just the grouping, as it creates a tree-like diagram, that shows these relationships.
- **Partitioning approaches:** The idea of partitioning approaches is to organize data into several groups based on a predefined criteria. This means that a "cost" function is introduced which is iteratively minimized until a satisfactory clustering is achieved.

9.1 k-means Clustering

The idea behind K-means clustering is to divide a given dataset into K distinct, non-overlapping clusters based on the features present in the data. The goal is to minimize the distances between data points and their corresponding cluster center.

1. Represent each of the k clusters by one single point μ_i for $i \in \{1, \dots, k\}$ (which denotes its center)
2. Assign datapoints to the closest center μ_i

Data is given as $D = \{x_1, x_2, \dots, x_n\}$, $x_j \in \mathbb{R}^d$ and each x_j is assigned to the closest cluster (closest cluster $= \operatorname{argmin}_i \|x_j - \mu_i\|_2$).



The centers are picked in such a way that the sum of all squared distances is minimized. The squared distance is used in order to give more weight to outliers, which makes the final solution more robust in case the input is noisy.

Finding the optimal solution is NP hard and non-convex. This means that there are local minima, where the optimization algorithm gets stuck in suboptimal solutions.

Definition

Lloyd's Heuristic:

- Initialize centers $\mu_1, \dots, \mu_k$ randomly or using another approach
- Assign each data point to the nearest center by calculating the distance between each point and each center and assigning each point to the closest center
- Update each center to be the average of the data points assigned to its cluster:

$$\forall i : \mu_i^{\text{new}} = \frac{1}{n_i} \sum_{x_i \text{ assigned to } \mu_i} x_i$$
- Repeat steps 2 and 3 until one of the below conditions is met:
 - The centers no longer change between iterations
 - The assignments of the datapoints no longer change between iterations
 - A maximum number of iterations is reached

Runtime: $O(nkd)$

There are a few drawbacks of Lloyd's Heuristic:

- The algorithm is sensitive to the initial selection of the centers. Different initializations lead to different solutions. Therefore, suboptimal solutions might be reached (in case the algorithm optimized towards a local minimum)
- k-means assumes that clusters are spherical (or intersected by other spheres), which might not be optimal for some datasets. Therefore, it should not be applied for clusters with shapes other than spheres.
- k-means requires the number of clusters to be specified in advance, which might not be known before.

9.2 k-means++ Clustering

k-means++ is an alternative to the random initialization of the centers $\mu_1, \dots, \mu_k$.

Definition

k-means++:

1. Start with a random point as the first center: $\mu_1 = x_i$ where $i \sim \text{Unif}(1, \dots, n)$
2. For each **datapoint** x not chosen yet, compute the distance between x and the nearest center. Choose one new data point at random as a new center, using a weighted probability distribution where a point x is chosen with probability proportional to the distance to its nearest center. This makes datapoint far away from centers more likely to be chosen as new centers.
3. Repeat steps 2 until k centers have been chosen.

By biasing the selection of initial centers towards points that are far apart from each other, k-means++ leads to a wider spread of the initial centers and reduces the likelihood of converging to suboptimal solutions. Also, it has been shown that the algorithm's expected "loss function value" (sum of squared distances) is at most $O(\log(k))$ times the optimal value (k being the number of clusters).

10 Unsupervised learning: Dimensionality reduction

Definition

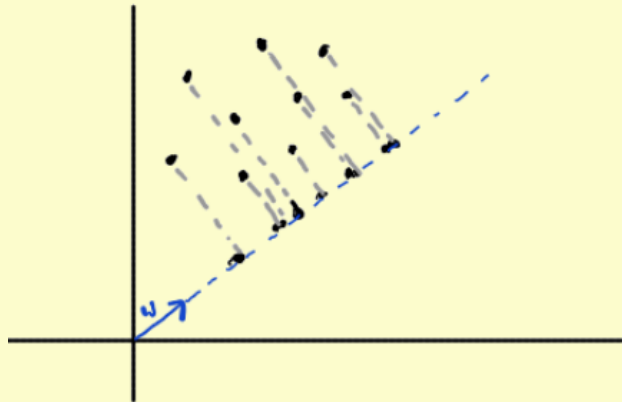
Dimensionality Reduction: Given a data set $D = \{x_1, x_2, \dots, x_n\} \in \mathbb{R}^d$, obtain an "embedding" (low-dimensional representation of the same data) $\{z_1, z_2, \dots, z_n\} \in \mathbb{R}^k$ where k is much smaller than d .

The goal is to find a mapping $\mathbb{R}^d \rightarrow \mathbb{R}^k$ which we can use to create the embeddings. We distinguish between:

- Linear dimensionality reduction ($f(x) = Ax$)
- Non-Linear dimensionality reduction

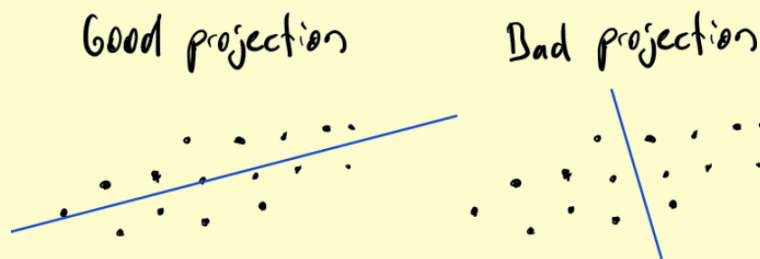
Clarification

Example: Let's perform a simple dimensionality reduction from $d = 2$ to $k = 1$.



We want to transform the 2-dimensional data into 1-dimensional data (a line). We first need to make sure that the data is centered ($\mu = \frac{1}{n} \sum_i x_i = 0$). If it is not centered then we need to make it centered by calculating μ and subtracting it from all x_i .

We want to project the points x_i onto the line w . Then we just need a 1-dimensional coefficient to express every point on that line ($x_i \approx z_i w$ where $w \in \mathbb{R}^2$ represents the line and $z_i \in \mathbb{R}^1$ the location of the projection on the line). Our aim is to identify the w that best explains the data D .



In other words, we want to minimize $\|z_i w - x_i\|_2^2$ for all points. Additionally, we enforce $\|w\|_2 = 1$ (without loss of generality). w defines the projection line.

$$(w^*, z^*) = \operatorname{argmin}_{\{w, z_1, \dots, z_n\}, \|w\|_2=1} \sum_{i=1}^n \|z_i w - x_i\|_2^2$$

In our $k = 1$ case, the optimal z is given by $z_i^* = w^T x_i$ (orthogonal projection of x_i on w). Remember that z_i is the factor by which we scale the normalized w vector to end up at the projection of point x_i .

This means that $z_i w$ becomes $w^T x_i w$ (since $z_i = w^T x_i$). $w^T x_i w$ are the coordinates of the projected point. This makes it possible to optimize just over w instead of both w and z :

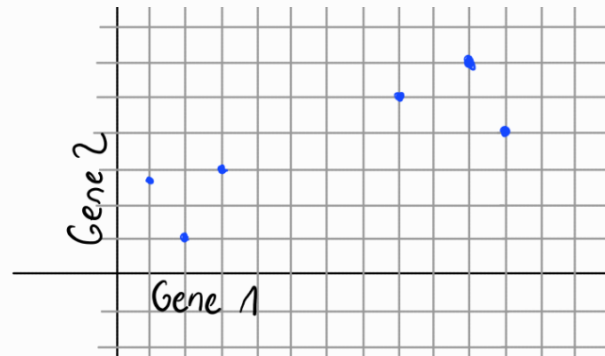
$$w^* = \operatorname{argmin}_{w, \|w\|_2=1} \sum_{i=1}^n \|w^T x_i w - x_i\|_2^2$$

10.1 Principal Component Analysis

The goal is to reduce the dimensionality of the data. Below is the procedure with an example. Let's consider this dataset:

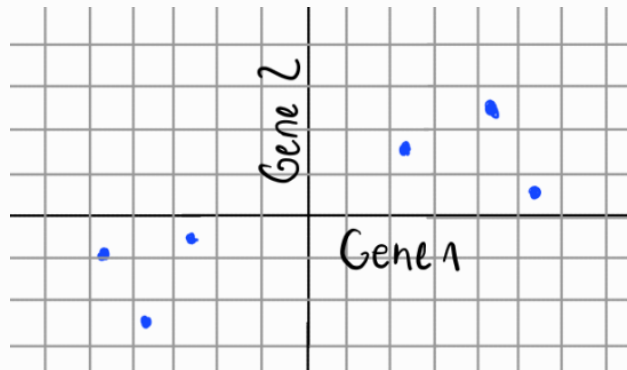
	Mouse 1	Mouse 2	Mouse 3	Mouse 4	Mouse 5	Mouse 6
Gene 1	10	11	8	3	1	2
Gene 2	6	4	5	3	2.8	1

The corresponding plot looks like this:



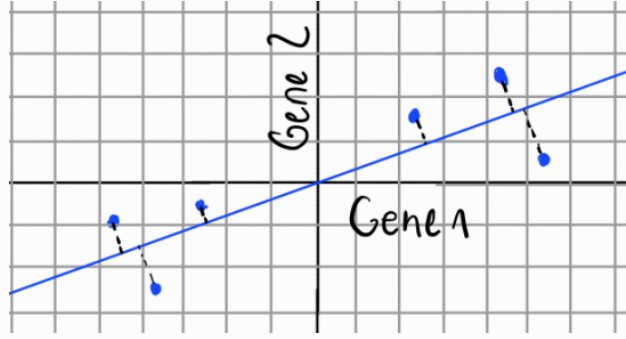
Let $x_1 = \begin{pmatrix} 10 \\ 6 \end{pmatrix}$, $x_2 = \begin{pmatrix} 11 \\ 4 \end{pmatrix}$ etc., and $n = 6$ our number of samples.

1. The first step is to center the data. This means we make sure that $\frac{1}{n} \sum_{i=1}^n x_i = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$ holds. This is not the case for our example, as $\frac{1}{n} \sum_{i=1}^n x_i = \begin{pmatrix} 5.8333 \\ 3.6333 \end{pmatrix}$. Therefore we subtract $\begin{pmatrix} 5.8333 \\ 3.6333 \end{pmatrix}$ from every sample.



Now we shifted all data points such that their average is equal to the null vector. Shifting the data did not change how the data points are positioned relative to each other.

2. As we want to bring the data points to one dimension, we are looking for a line that best captures the variance of the data points. The data points are projected to the line and the line goes through the origin.



We want to find a line that maximizes the distances of the projected points to the origin. This is because we want to capture as much variance as possible.

3. We calculate the covariance matrix S .

For simplicity, let's transform the dataset to this matrix:

$$X = \begin{bmatrix} 10 & 6 \\ 11 & 4 \\ 8 & 5 \\ 3 & 3 \\ 2 & 2.8 \\ 1 & 1 \end{bmatrix} - \begin{bmatrix} 5.8333 & 3.6333 \\ 5.8333 & 3.6333 \\ 5.8333 & 3.6333 \\ 5.8333 & 3.6333 \\ 5.8333 & 3.6333 \\ 5.8333 & 3.6333 \end{bmatrix} = \begin{bmatrix} 4.1666 & 2.3666 \\ 5.1666 & 0.3666 \\ 2.1666 & 1.3666 \\ -2.8333 & -0.6333 \\ -4.8333 & -0.8333 \\ -3.8333 & -2.6333 \end{bmatrix}.$$

$$\begin{aligned} S &= \begin{pmatrix} \text{COV}(G1, G1) & \text{COV}(G1, G2) \\ \text{COV}(G2, G1) & \text{COV}(G2, G2) \end{pmatrix} && / \text{ def of COV-matrix} \\ &= \frac{1}{n} \begin{pmatrix} \sum_{i=1}^n (x_{i;1} - \mu_{G1})^2 & \sum_{i=1}^n (x_{i;1} - \mu_{G1})(x_{i;2} - \mu_{G2}) \\ \sum_{i=1}^n (x_{i;2} - \mu_{G2})(x_{i;1} - \mu_{G1}) & \sum_{i=1}^n (x_{i;2} - \mu_{G2})^2 \end{pmatrix} && / \text{ def of COV} \\ &= \frac{1}{n} \begin{pmatrix} \sum_{i=1}^n (x_{i;1})^2 & \sum_{i=1}^n (x_{i;1})(x_{i;2}) \\ \sum_{i=1}^n (x_{i;2})(x_{i;1}) & \sum_{i=1}^n (x_{i;2})^2 \end{pmatrix} && / \mu_{G1} = 0, \mu_{G2} = 0 \text{ (centered)} \\ &= \frac{1}{n} X^T X \\ &= \begin{pmatrix} 15.805 & 5.105 \\ 5.105 & 2.605 \end{pmatrix} \end{aligned}$$

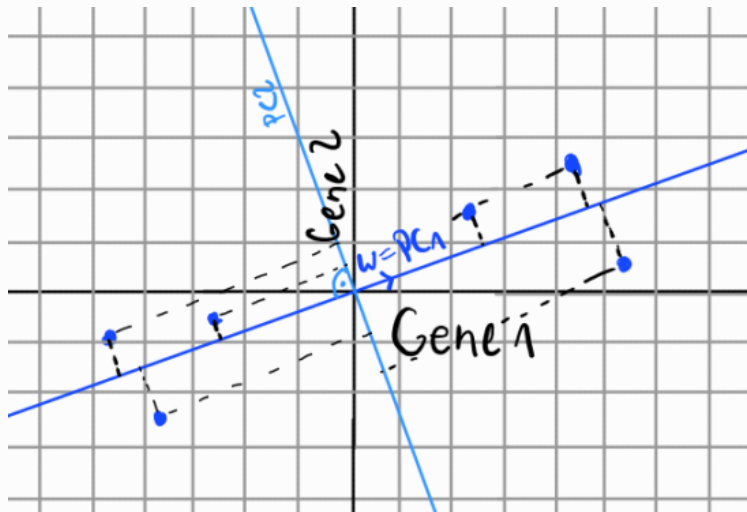
S is symmetric because $\text{COV}(A, B) = \text{COV}(B, A)$. The diagonal elements of S are the variance of the features.

4. Now we need to calculate the normalized eigenvalues and eigenvectors of S . The results are:

$$v_1 = \begin{pmatrix} -0.323 \\ 0.946 \end{pmatrix}, \lambda_1 = 0.861,$$

$$v_2 = \begin{pmatrix} 0.946 \\ 0.323 \end{pmatrix}, \lambda_2 = 17.55$$

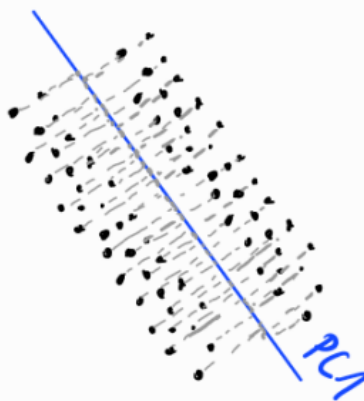
The eigenvector with highest eigenvalue is the first principal component.



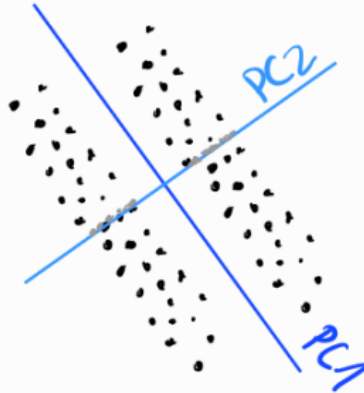
Clarification

- The eigenvalue is equivalent to the average sum of squared distances from the projected points to the origin ($\frac{1}{n}(Xv_2)^T Xv_2 = \lambda_2$).
- If we consider k principal components and our data was originally in dimension d , then the overall loss is $n \sum_{i=k+1}^d \lambda_i$.
- There is also a connection to the singular value decomposition: when decomposing X to $X = U\Sigma V^T$ (U, V are unitary). The top k principal components are the first k columns of V .
- Principal components are always orthogonal to each other, because eigenvectors are orthogonal to each other as well. Intuition: Principal components are defined to capture the maximum variance in the data and orthogonality ensures that each principal component describes a unique direction in the data.

It is important to note that PCA is not always useful when it comes to classification:



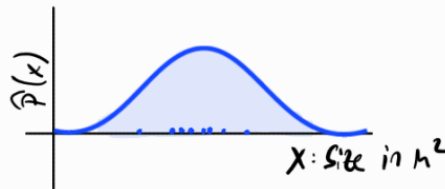
The second principal component would be much better for solving the classification problem:



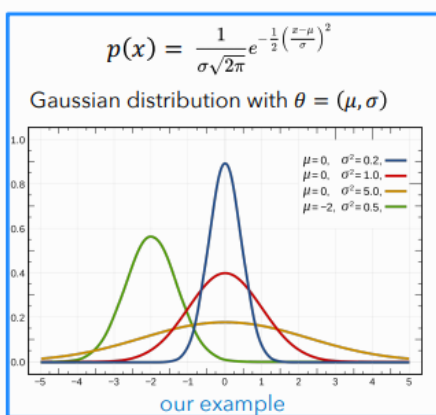
11 Probabilistic modeling

The idea is to not find the best fitting function $\hat{f} \in F$, but to find the probability distribution of the unseen data $\hat{\mathbb{P}} \in P$. Thereby we assume that the training data has the same distribution as the unseen data. Some applications of probabilistic modeling are:

- conditioned on previous sentences, generate the next word
- given artist and style, generate a new song
- find certain parameters (e.g. mean, variance) that describe nature or society
- detect atypical x (anomaly and out-of-distribution detection)

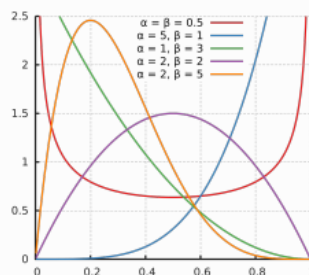


The first step is to specify a probability class P . For $d = 2$ (so 2 parameters), this could be one of the below:



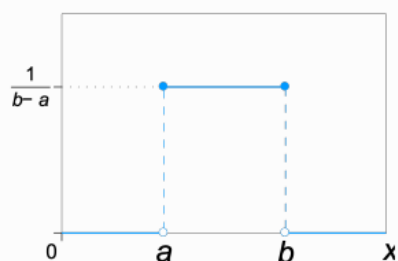
$$p(x) = \frac{1}{B(\alpha, \beta)} x^{\alpha-1} (1-x)^{\beta-1}$$

beta distribution with $\theta = (\alpha, \beta)$



$$p(x) = \begin{cases} \frac{1}{b-a} & \text{for } a \leq x \leq b, \\ 0 & \text{for } x < a \text{ or } x > b. \end{cases}$$

Uniform distribution with $\theta = (a, b)$



There are two fundamental opposite paradigms in statistics:

Definition

Frequentist Approach: In the frequentist view, the true distribution is fixed.

- **Probability:** Probability is interpreted as the long-run frequency of events. Example: the probability of a coin landing heads is the proportion of heads observed in a large number of coin flips.
- **Parameters:** Parameters (e.g. mean or variance) are considered fixed but unknown quantities. They are not random variables.
- **Methods:** Maximum-likelihood estimation, confidence intervals, hypothesis testing

Definition

Bayesian Approach: In the Bayesian view, the true distribution is a random draw.

- **Probability:** In the Bayesian view, probability is a measure of belief or certainty about an event. It is a matter of personal perspective and can change with new evidence.
- **Parameters:** Parameters are treated as random variables with their own probability distributions (prior distributions).
- **Methods:** Maximum-a-posteriori estimation

Let's take a look at both methods.

11.1 Maximum-likelihood estimation (Frequentist)

Let's consider searching in a parametric probability class P . This class ideally includes the "ground truth" probability distribution $\mathbb{P}$. The idea behind the maximum-likelihood approach is to find the distribution $\mathbb{P} \in P$ under which the observed data samples have the highest probability ("likelihood") of being observed (compared to any other $\mathbb{P} \in P$).

The likelihood is expressed by $P((x_1, \dots, x_n), \Theta)$, where $(x_1, \dots, x_n)$ are the data samples and Θ are the parameters of the probability distribution.

For a fixed probability distribution $\mathbb{P}_\Theta \in P$: if each training sample is independent and identically distributed ("iid") with $x_i \sim \mathbb{P}_\Theta$, then the random dataset $D = \{x_i\}_{i=1}^n$ has a joint distribution of $P(D; \Theta) = p((x_1, \dots, x_n), \Theta) = \prod_{i=1}^n p(x_i; \Theta)$ (factorization because of independence).

In practice we use the log-maximum-likelihood, because it allows us to rewrite products into sums

$(\log(a \cdot b) = \log(a) + \log(b))$. We are allowed to take the log because taking the log is a monotonic transformation. This means it preserves the location of the maximum, meaning that the maximum of the likelihood function and the maximum of its logarithm occur at the same parameter values.

$$\begin{aligned}
 \hat{\Theta}_{MLE} &= \operatorname{argmax}_{\Theta} (p(D; \Theta)) && / \text{ we want maximum probability} \\
 &= \operatorname{argmax}_{\Theta} (\log(p(D; \Theta))) && / \text{ log doesn't change optimal parameters} \\
 &= \operatorname{argmax}_{\Theta} (\log(\prod_{i=1}^n p(x_i; \Theta))) && / \text{ assuming iid} \\
 &= \operatorname{argmax}_{\Theta} \left(\sum_{i=1}^n \log(p(x_i; \Theta)) \right) && / \text{ log laws}
 \end{aligned}$$

Clarification

Example: MLE for Gaussian distribution

- Given dataset: $D = \{(x_1, y_1), \dots, (x_n, y_n)\}$
- MLE for class label probability: $p(Y = y) = \frac{\#Y=y}{n} = \hat{p}_y$
- MLE for feature distribution:
 - $p(x|y) = N(x; \hat{\mu}_y, \hat{\Sigma}_y)$
 - $\hat{\mu}_y = \frac{1}{\#Y=y} \sum_{i: y_i=y} x_i$
 - $\hat{\Sigma}_y = \frac{1}{\#Y=y} \sum_{i: y_i=y} (x_i - \hat{\mu}_y)(x_i - \hat{\mu}_y)^T$ (covariance matrix)

Theorem

Bayes' Theorem:

- $p(a|b) = \frac{p(a;b)}{p(b)}$
- Therefore: $p(b|a) = \frac{p(a|b) \cdot p(b)}{p(a)}$
- The joint probability is defined as $p(x, y) = p(x)p(y|x)$

11.2 Maximum-a-posteriori estimation (Bayesian)

Maximum-a-posteriori estimation (MAP) can also be used to estimate the unknown parameters of a probability distribution. It is an extension of the the maximum-likelihood estimation which involves prior information about the parameter through Bayes' theorem. So if we have any prior information about the parameters, then we can use MAP.

$$\begin{aligned}
 \Theta_{MAP} &= \operatorname{argmax}_{\Theta} (p(D; \Theta)) && / \text{ we want maximum probability} \\
 &= \operatorname{argmax}_{\Theta} (p(D|\Theta)p(\Theta)) && / \text{ Bayes' Theorem}
 \end{aligned}$$

Here, $p(\Theta)$ encodes any prior knowledge or beliefs about the parameter.

11.3 Discriminative vs. Generative approach

There are two approaches

- **Discriminative:** The main goal is to estimate the conditional probability $P(y|x)$ (y is the label and x is the input feature vector). We are only interested in finding $p(y|x)$. One example might be: given nutritional values (e.g. calories, fat, protein, carbohydrates, etc.), is a meal healthy or not?
 → Simple to compute $p(y|x)$

- **Generative:** The main goal is to understand how the data was generated. We aim to find the joint probability $p(x, y)$ (which is equal to $p(x|y)p(y)$). This means that we also learn the probability distribution of x given a y , which means we can reconstruct the input features if we know a label y . One example might be: We identified someone with a disease (e.g. flu, covid, cold, etc.) and we want to reconstruct the symptoms (e.g. loss of taste, headache, etc.). Another example is creating images of digits based on the input $y = 1, y = 2$, etc. → Simple to compute $p(x|y)$

Both approaches can give us the joint probability $p(x, y)$. The difference is that discriminative models parameterize $p(y|x)$ and generative models parameterize $p(x|y)$.

11.4 Gaussian Mixture Models

The idea behind Gaussian Mixture Models (GMMs) is to represent a complex distribution of data as a combination of multiple Gaussian distributions. Each Gaussian component in the mixture represents a cluster or subgroup within the overall population.



- A d -dimensional Gaussian distribution is defined by its mean vector $\mu \in \mathbb{R}^d$ and the $d \times d$ -dimensional covariance matrix Σ
- In a mixture model, the probability distribution of data points is modeled as a combination of different component distributions. For GMMs, these components are Gaussian distributions.
- The different Gaussian distributions are weighted to represent the probability of data points being generated by that component. The weights sum up to 1 (for GMM with k distributions: $\sum_{i=1}^k w_i = 1$).

This results to:

$$p(x) = \sum_{i=1}^k w_i N(x|\mu_i, \Sigma_i)$$

The parameters of a GMM are typically estimated using the expectation-maximization algorithm:

- w_k : The weight of every Gaussian component
- μ_k : Every Gaussian component has one mean vector μ_k
- $\Sigma_{k \times k}$: Every Gaussian component has one covariance matrix $\Sigma_{k \times k}$

Definition**Expectation-maximization (EM) algorithm:**

1. **Initialization:** Start with initial guesses for the parameters. If the GMM consists of k Gaussian distributions, then we have k parameters $\Theta_1, \dots, \Theta_k$ where $\Theta_j = (w_j, \mu_j, \Sigma_j)$ and:
 - w is a scalar representing the weight of distribution j
 - μ_j is a $d \times 1$ vector containing the means of distribution j
 - Σ_j is a $d \times d$ covariance matrix of distribution j
2. **Expectation Step (E):** For each point, calculate the probability of that point being in the different Gaussian components (calculate the "responsibilities"). This means, calculate the probability that point x_i is covered by component j :

$$\gamma_{i,j} = \frac{w_j N(x_i | \mu_j, \Sigma_j)}{\sum_{o=1}^k w_o N(x_i | \mu_o, \Sigma_o)}$$

3. **Maximization Step (M):** Update the parameters based on the current responsibilities:

$$w_j^{new} = \frac{1}{n} \sum_{i=1}^n \gamma_{i,j}$$

$$\mu_j^{new} = \frac{\sum_{i=1}^n \gamma_{i,j} x_i}{\sum_{i=1}^n \gamma_{i,j}}$$

$$\Sigma_j^{new} = \frac{\sum_{i=1}^n \gamma_{i,j} (x_i - \mu_j^{new})(x_i - \mu_j^{new})^T}{\sum_{i=1}^n \gamma_{i,j}}$$

4. **Iteration:** Repeat step 2 and 3 until convergence (i.e. parameters do not change significantly anymore).

The initialization can happen as follows:

- **Weights:** Uniform distribution ($w_j = \frac{1}{k}$)
- **Means:** Random initialization but with k-means++ procedure
- **Variances:** Initialize as spherical, according to empirical variance in the data (the distributions should cover all the data)